EUR/USD ORACLE - Implementation Atlas, Development Ledger and Research-First Master Plan

Reference edition: 1.0 | **As of:** 30 October 2026 | **Research operating decision:** FINAL=HOLD
Latest user-verified Git state: feature/utc-research-m01-m08 at fb1c643 (local and origin/ tracking heads matched; clean tree in supplied PowerShell log)
Next proposed work: Stage 6, Batch 1 (P6-01 to P6-04) - research governance and evidence contracts. **Not yet started.**

Document contents (navigation)

For the complete code reference, use Section 12. The evidence gates and Stage 6 plan are in Sections 7 and 10.

How to use this reference and what was actually examined

Executive current-state statement - read this first

The system as implemented versus the intended closed loop

Folder-level orientation

The Council of 31 - exact roles, categories and inputs

Development history and exact checkpoint ledger

Proofs, evidence gaps and the difference between tests and truth

Unfinished implementation, risks and safeguards

Original Blueprint phases versus engineering Stage 6 - do not confuse these

Full planned development program from this checkpoint

Safe operating and resumption instructions for the next developer/chat

Directly inspected October 27 source inventory - file-by-file

Post-archive Stage 2-5 implementation additions - separate provenance

Non-Python archive artifacts and support material

Exact transition point and next action

Complete auxiliary model and audit-file register (archive only)

1. How to use this reference and what was actually examined

This is the continuity document for a new developer, auditor, or ChatGPT conversation. Read Sections 1-6 before proposing changes; use the detailed file-by-file atlas in Section 12 for navigation. Update the development ledger and evidence register after every future batch; do not overwrite historical completion evidence.

Evidence tiers are deliberately separated:

A - directly inspected source snapshot: EURUSD_ORACLE.zip, a supplied October 27, 2026 project archive. Its first-party backend/ contains **316 Python files**: **206 non-test modules, including empty initializers**, and **110 test modules**. All 316 were read for structural inventory and successfully parsed as Python AST; key control-flow modules and milestone notes were additionally examined at source level. This is **static review**, not a claim of line-by-line formal verification or running the archived code.

B - directly available later code: the final Stage 5 prospective evidence inspector and final handover module/test files in the supplied local workspaces were available to inspect, alongside the full Stage 5 source presented in the project conversation. They are **not** present in the older ZIP.

C - user-supplied terminal evidence and development history: Windows test reports, real read-only inspection outputs and Git commit/push logs support Stages 3-5 status. Those logs are reliable accounts of observed commands, **not an independent live checkout of commit fb1c643 in this environment**.

D - intended architecture: the original seven-page Blueprint v1.1 and the five-page Research-First v1.2 amendment specify the desired future operating model. Roadmap commitments are not implemented features merely because they appear in a blueprint.

The archive includes .env, Git internals, a virtual environment, compiled caches, audit logs and archived model artifacts. **No credential values are reproduced in this document.** Sensitive .env files and private broker/account identifiers must never be committed or shared for documentation. No MT5 process, database collection, Council run, training, settlement or trading operation was started to prepare this reference.

Currency of the inventory: Section 12 describes the available October 27 archive, not the entire exact October 30 branch. Section 13 separately describes the subsequently implemented Stage 4 and Stage 5 modules from verified engineering records. A current-branch, byte-level reconciliation will require a fresh safe source archive or local repository inventory produced at fb1c643 or later. Do not treat absent paths in the October 27 archive as proof they are absent today.

2. Executive current-state statement - read this first

- **Original Council architecture** — **Evidence-backed status at Stage 5 close:** Canonical manifest and registered implementations for **31 seats**; primary -> governance -> strategist orchestration; research/synthetic integration tests exist **What is not established:** Predictive edge, independent votes, fresh trusted input availability, permitted live Council operation
- **Market data and UTC feed** — **Evidence-backed status at Stage 5 close:** MT5 adapters, storage, supervision, source-bound clock policy and isolated forward UTC tables exist. Historical

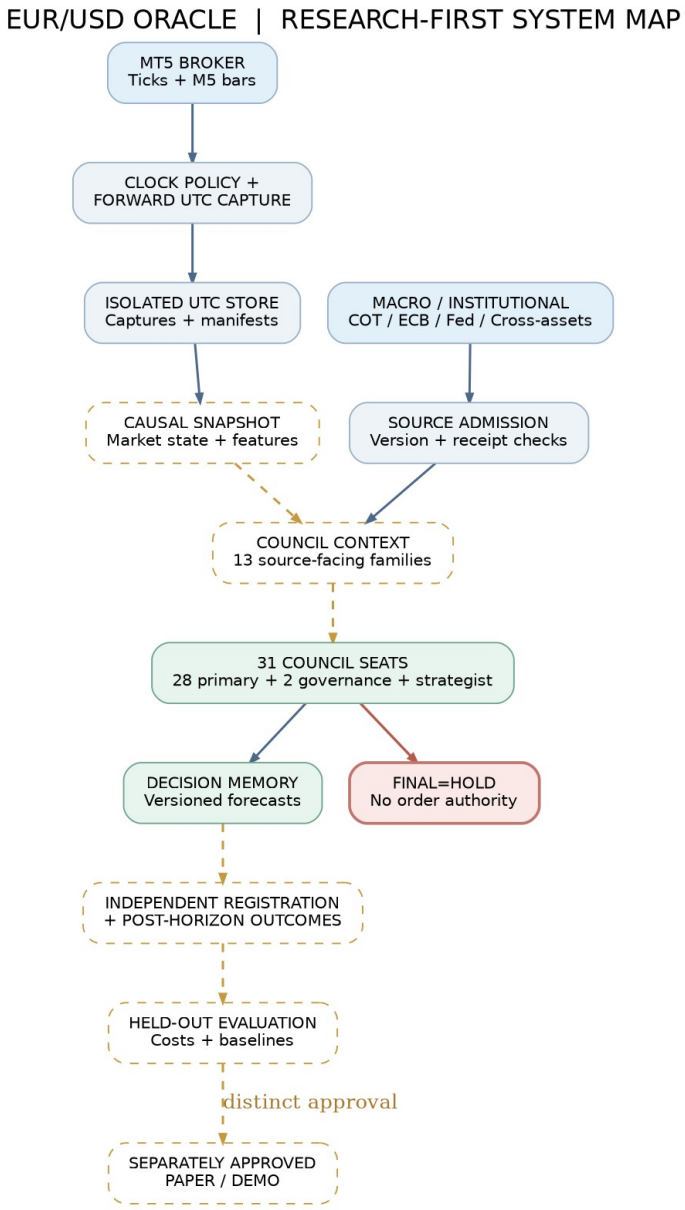
snapshot/inspection: **10 capture receipts, 1,507 first-owned ticks, six completed M5 bars What is not established:** Continuous currently approved forward session, independently authenticated broker offset, historical legacy UTC certification

- **Market-state calculations** — **Evidence-backed status at Stage 5 close:** Trend, volatility, momentum, structure, liquidity and session engines **What is not established:** Proven fresh end-to-end feed into an autonomously scheduled Council
- **Historical quantitative models** — **Evidence-backed status at Stage 5 close:** Dataset/feature/label/split machinery; XGBoost, neural three-member ensemble and HMM artifact families and evaluation metadata **What is not established:** Independently verified profitable strategy; unspent test set; full walk-forward net-return validation
- **Institutional/macro sources** — **Evidence-backed status at Stage 5 close:** CFTC COT, ECB, Fed candidate/certification, calendar and cross-asset modules; strict source-admission safeguards **What is not established:** Complete independently timestamped and admitted point-in-time data for 13 Council source families; Fed production certification
- **Council memory** — **Evidence-backed status at Stage 5 close:** Append-only, idempotent decisions, forecasts and matured outcomes with explicit scopes **What is not established:** A populated independent prospective track record or automatically learned reliable weights
- **Stage 1-5 research engineering** — **Evidence-backed status at Stage 5 close:** Historical phases and completed S4/S5 engineering; Stage 5 **24/24** checkpoints committed, **847 passed / 9 skipped / 61 subtests passed** on user Windows regression **What is not established:** Software coverage is not market correctness, operational admission or profitability
- **Trading and dashboard** — **Evidence-backed status at Stage 5 close:** Governance/risk veto logic exists inside Council; CouncilRun.execution_authorized is false **What is not established:** Independent position/order lifecycle, authorized paper/live worker, dashboard, finished MiroFish-style environment
- **Evidence/admission** — **Evidence-backed status at Stage 5 close: 0/11** independent external gates verified; **0/13** source-facing families independently verified in Stage 5 handover **What is not established:** Any upgrade from FINAL=HOLD

Decision: Stage 5 engineering is complete. The project is **not research-operationally admitted**, not proven profitable, not running an authorized autonomous research loop, and not approved for paper or live execution. Stage 6 is proposed; Stage 6 code has not yet been committed.

3. The system as implemented versus the intended closed loop

3.1 Logical architecture



Important: solid boxes represent components and/or logical stage relationships in available source, **not proof that the full graph executes automatically today**. Dotted links are incomplete, gated, or proposed. The FINAL=HOLD boundary is always in force. A structural 31-seat run in isolated research does not equal an authorized production Council cycle.

3.2 Program flow and ownership

Broker/market ingress - backend/data_engine/mt5_provider.py, tick ingestors, candle ingestors and backend/runtime/capture_forward_feed.py retrieve/capture market observations. The forward path requires a bound clock policy and stores to separate UTC tables. The original local quote wall-label convention is **7,200 seconds**, not an independently proven UTC offset.

Persistence/quality - backend/database, backend/data_engine/forward_store.py and backend/data_quality define SQL models, checks, receipt manifests, deduplication and quality flags. Legacy market tables must not be casually treated as time-corrected.

Market state/features - backend/market_state produces structural, momentum, volatility, liquidity and session descriptions. backend/models produces historical datasets, labels, feature vectors and frozen model artifacts. Integration into a fresh approved forward epoch is not yet independently proven.

Economic/institutional data - backend/institutional and backend/macro capture/reconcile COT, central-bank and calendar data. Fed admission remains fail-closed and candidate data must not be promoted to trusted context by filename alone.

Council construction - backend/agents/council_manifest.py declares the exact 31 seats; register_core_agents.py, agent_registry.py, context_factory.py, contracts.py and execution_policy.py enforce registration, context validation and eligibility.

Council staged orchestration - council_runner.py executes **28 primary seats, two governance seats and one chief strategist**; blocked/error/missing conditions are not treated as successful votes. The strategy outputs research decisions only; execution_authorized=False.

Memory/settlement - backend/learning stores run provenance, versioned forecast specifications and delayed outcomes. It does not itself fetch trusted future quotes, retrain agents, authorize orders or certify profitable performance.

External evaluation and operating modes - Stages 4-5 added read-only forensic/attestation inspectors. A fully authenticated prospective ledger, acceptance process, paper execution, live execution and dashboard remain future work.

3.3 Data scope and central invariants

EURUSD, M5 and aware UTC timestamps are explicit assumptions in the current forward/evaluation work. Every source needs raw timestamps, normalized instants, first availability, capture receipt, provider identity, policy/code hash and revision/vintage provenance.

Original October 27 policy/epoch: 75311e239cd5c295d3e08e03c358488fb123a46bfc6d342dd31493d95710aa33; policy file docs/clock_profiles/hfm_forward_20261027.json; expected raw policy SHA-256 e34b7a29f78f5865f4b0fe7c4ad778e4e8b650d9943792eee019a48bdf2fe088. This is a historical anchor, **not a credential and not permission to rebind**.

Pinned original clock implementation SHA-256 03a1811446de521f5481024df2ef8572b51a46746d61ec1135f9f26e6cf60645 did **not** equal the Stage 5 Windows implementation; inspect changes and original source bytes before drawing conclusions. Historical archive line ending normalization and subsequent source changes must not be conflated.

Original capture manifests hash **complete ordered original tick/bar lists**. Later deduplicated, first-owned SQL rows cannot reconstruct the original lists. A genuine original-source file may allow local recomputation, **not** independent source authentication.

Local tests, source hashes and self-declared issuer/timestamp fields prove local consistency only; independently verified receipt origin and third-party time custody must be assessed separately.

Held-out splits and model versions should be frozen **before** the evaluated decisions; after-the-fact prediction entry, future macroeconomic actuals, retroactive revisions and post-horizon leaks are prohibited.

4. Folder-level orientation

- **backend/agents/** — **Responsibility:** Council contract, 31-seat inventory, registry, eligibility, source context, orchestration and research CLI **State / integration caution:** Registered seats do not imply eligible production source inputs or calibrated predictive value
- **backend/agents/implementations/** — **Responsibility:** Individual seat algorithms and governance agents **State / integration caution:** Diverse analytical methods and horizons are not yet empirically standardized for reliable weighted synthesis
- **backend/candle_engine/** — **Responsibility:** Legacy candle construction, storage, scheduler helpers and MT5 bar ingestion **State / integration caution:** Legacy timestamps must stay isolated from newly certified forward UTC domains
- **backend/config/** — **Responsibility:** Settings and runtime configuration **State / integration caution:** .env contains private settings; never publish it
- **backend/data_engine/** — **Responsibility:** Provider interfaces, MT5 adapters, ticks, forward UTC epoch/storage and data checks **State / integration caution:** Legacy and UTC feed paths are different; clock policy is pinned
- **backend/data_quality/** — **Responsibility:** Quality events and market-data validation **State / integration caution:** Checks do not replace external source custody
- **backend/database/** — **Responsibility:** SQL connectivity, schemas/models, UTC SQL conversions and indexes **State / integration caution:** DB schemas are not equivalent to a live, currently working connection
- **backend/institutional/** — **Responsibility:** CFTC TFF COT acquisition, publication calendar, snapshots, positioning features and research splits **State / integration caution:** COT releases need causal date/receipt tracking
- **backend/learning/** — **Responsibility:** Append-only Council memory, forecast specifications, scoped outcome settlement **State / integration caution:** No continuous independently attested prospective ledger or adaptive weighting yet
- **backend/macro/** — **Responsibility:** ECB, Fed/FOMC, ALFRED, calendars, rate storage and cross-asset capture **State / integration caution:** Fed real source certification and 13-family admission remain blocked
- **backend/market_state/** — **Responsibility:** Price trend, structure, momentum, volatility, liquidity and sessions **State / integration caution:** Historical feature computation must not silently be treated as forward UTC-certified
- **backend/models/** — **Responsibility:** Dataset creation, feature/label contracts, training, registries, evaluations and frozen model artifacts **State / integration caution:** Historical ML evaluations are not economic profits; original holdout is already inspected
- **backend/runtime/** — **Responsibility:** Supervisor, process locks, MT5 health checks, time-domain probe, forward feed recorder; later Stage 2-5 read-only inspectors **State / integration caution:** Some runtime commands can connect to MT5 or write DB; the Stage 5 inspection commands deliberately do not
- **docs/** — **Responsibility:** Dated engineering audit/handover records and original clock policy/probe **State / integration caution:** Older notes describe historical states and can be superseded by later commits

- **logs/** — **Responsibility:** Test logs, source snapshots, local audit reports, synthetic fixtures and investigation scripts **State / integration caution:** Historical diagnostics, not a current broker performance audit; never add raw logs/credentials indiscriminately to Git
- **venv/** — **Responsibility:** Bundled third-party Python environment and binaries **State / integration caution:** Exclude from source review and version control; Windows-specific, not portable
- **.git/, .pytest_cache/, __pycache__/** — **Responsibility:** Repository metadata, test cache, compiled bytecode **State / integration caution:** Generated/internal material; not application behavior
- **.env, .env.example, requirements*.txt** — **Responsibility:** Private configuration, public setting names and dependency declarations **State / integration caution:** Never reproduce secret contents; locked Windows dependencies may require separate platform support

5. The Council of 31 - exact roles, categories and inputs

The following seat inventory is extracted from the archived COUNCIL_MANIFEST, not invented from agent names. The original blueprint groups seats 1-8 technical, 9-16 quantitative, 17-23 institutional, 24-28 macro and 29-31 governance. Implementation categories can further distinguish statistical versus machine-learning seats.

- **1** — **Agent:** Trend Master (trend_master) **Blueprint group:** Technical **Manifest purpose:** Evaluate persistent directional trend conditions. **Manifest required inputs:** market_state, candles
- **2** — **Agent:** Price Action (price_action) **Blueprint group:** Technical **Manifest purpose:** Evaluate recent candle behaviour, rejection and directional pressure. **Manifest required inputs:** candles
- **3** — **Agent:** Support / Resistance (support_resistance) **Blueprint group:** Technical **Manifest purpose:** Evaluate price relative to meaningful support and resistance zones. **Manifest required inputs:** candles
- **4** — **Agent:** Momentum (momentum) **Blueprint group:** Technical **Manifest purpose:** Evaluate current directional force and momentum acceleration. **Manifest required inputs:** market_state, candles
- **5** — **Agent:** Volatility (volatility) **Blueprint group:** Technical **Manifest purpose:** Evaluate the current volatility regime and volatility-adjusted opportunity. **Manifest required inputs:** market_state, candles
- **6** — **Agent:** Fibonacci (fibonacci) **Blueprint group:** Technical **Manifest purpose:** Evaluate retracement and extension relationships around confirmed swings. **Manifest required inputs:** candles, market_structure
- **7** — **Agent:** Pattern Recognition (pattern_recognition) **Blueprint group:** Technical **Manifest purpose:** Detect recurring candle and geometric price patterns. **Manifest required inputs:** candles
- **8** — **Agent:** Market Structure (market_structure) **Blueprint group:** Technical **Manifest purpose:** Evaluate confirmed swing structure and structural transitions. **Manifest required inputs:** market_state, candles
- **9** — **Agent:** Statistical Probability (statistical_probability) **Blueprint group:** Quantitative **Manifest purpose:** Estimate direction from empirical conditional probabilities. **Manifest required inputs:** candles, market_state
- **10** — **Agent:** Monte Carlo (monte_carlo) **Blueprint group:** Quantitative **Manifest purpose:** Estimate directional outcome distributions through simulation. **Manifest required inputs:** candles, market_state
- **11** — **Agent:** ML XGBoost / Random Forest (ml_tree_ensemble) **Blueprint group:** Quantitative **Manifest purpose:** Generate predictions from validated tree-based machine-learning models. **Manifest required inputs:** feature_vector, model_artifact

- **12** — **Agent:** Neural Network (neural_network) **Blueprint group:** Quantitative **Manifest purpose:** Generate nonlinear predictions from validated neural models. **Manifest required inputs:** feature_vector, model_artifact
- **13** — **Agent:** Hidden Markov Model (hidden_markov_model) **Blueprint group:** Quantitative **Manifest purpose:** Estimate latent market regime and regime-transition probabilities. **Manifest required inputs:** candles, model_artifact
- **14** — **Agent:** Mean Reversion (mean_reversion) **Blueprint group:** Quantitative **Manifest purpose:** Estimate whether price displacement is statistically likely to revert. **Manifest required inputs:** candles, market_state
- **15** — **Agent:** Time Series (time_series) **Blueprint group:** Quantitative **Manifest purpose:** Evaluate temporal dependence and short-horizon directional forecasts. **Manifest required inputs:** candles
- **16** — **Agent:** Anomaly Detection (anomaly_detection) **Blueprint group:** Quantitative **Manifest purpose:** Detect abnormal market behaviour that may invalidate ordinary models. **Manifest required inputs:** candles, market_state
- **17** — **Agent:** Liquidity Hunter (liquidity_hunter) **Blueprint group:** Institutional **Manifest purpose:** Evaluate liquidity concentration, sweeps and likely liquidity targets. **Manifest required inputs:** liquidity_state, market_structure, tick_features
- **18** — **Agent:** Smart Money (smart_money) **Blueprint group:** Institutional **Manifest purpose:** Evaluate displacement, liquidity behaviour and institutional-style structure. **Manifest required inputs:** candles, market_structure, tick_features
- **19** — **Agent:** Order Flow (order_flow) **Blueprint group:** Institutional **Manifest purpose:** Evaluate tick-level directional pressure and microstructure imbalance. **Manifest required inputs:** tick_features, liquidity_state
- **20** — **Agent:** London Session (london_session) **Blueprint group:** Institutional **Manifest purpose:** Evaluate EUR/USD behaviour in the London trading session. **Manifest required inputs:** session_state, candles
- **21** — **Agent:** New York Session (new_york_session) **Blueprint group:** Institutional **Manifest purpose:** Evaluate EUR/USD behaviour in the New York trading session. **Manifest required inputs:** session_state, candles
- **22** — **Agent:** COT / Institutional Positioning (cot_positioning) **Blueprint group:** Institutional **Manifest purpose:** Evaluate institutional positioning from validated positioning data. **Manifest required inputs:** cot_data, market_state
- **23** — **Agent:** Dealer Behaviour (dealer_behaviour) **Blueprint group:** Institutional **Manifest purpose:** Evaluate dealer-style inventory and microstructure behaviour. **Manifest required inputs:** tick_features, market_state
- **24** — **Agent:** ECB (ecb) **Blueprint group:** Macroeconomic **Manifest purpose:** Evaluate ECB policy and euro-side monetary conditions. **Manifest required inputs:** ecb_data, economic_calendar
- **25** — **Agent:** Federal Reserve (fed) **Blueprint group:** Macroeconomic **Manifest purpose:** Evaluate Federal Reserve policy and US dollar monetary conditions. **Manifest required inputs:** fed_data, economic_calendar
- **26** — **Agent:** Rate Differential (rate_differential) **Blueprint group:** Macroeconomic **Manifest purpose:** Evaluate relative euro and dollar interest-rate conditions. **Manifest required inputs:** rates_data

- **27** — **Agent:** Economic Calendar (economic_calendar) **Blueprint group:** Macroeconomic **Manifest purpose:** Evaluate scheduled macroeconomic event risk and surprise information. **Manifest required inputs:** economic_calendar
- **28** — **Agent:** Risk Sentiment (risk_sentiment) **Blueprint group:** Macroeconomic **Manifest purpose:** Evaluate broader cross-asset risk-on and risk-off conditions. **Manifest required inputs:** cross_asset_data, market_state
- **29** — **Agent:** Devil's Advocate (devils_advocate) **Blueprint group:** Governance **Manifest purpose:** Challenge dominant Council reasoning and search for contradictory evidence. **Manifest required inputs:** primary_agent_opinions, market_state
- **30** — **Agent:** Risk Officer (risk_officer) **Blueprint group:** Governance **Manifest purpose:** Evaluate whether current conditions justify taking market risk. **Manifest required inputs:** primary_agent_opinions, market_state, risk_state
- **31** — **Agent:** Chief Strategist (chief_strategist) **Blueprint group:** Governance **Manifest purpose:** Synthesize analytical and governance opinions into the final Council view. **Manifest required inputs:** primary_agent_opinions, governance_opinions, market_state

Council safeguards: Production readiness is not conferred by registry membership. execution_policy.py applies live/source eligibility requirements; the Fed and rate-differential path lacks certified Fed data, and the production calendar path is blocked without admission. The Risk Officer requires explicit limits and portfolio state. The Devil's Advocate records conflicting evidence. The Chief Strategist can return HOLD. A completed research Council run still has **no execution authorization**.

6. Development history and exact checkpoint ledger

6.1 Pre-Stage-1 engineering represented in the supplied archive (October 2026)

The archived source and dated docs/ checkpoints show database/tick/candle ingestion, market state, 31-seat Council consolidation, historical ML experiment and registry work, Fed provenance defense, memory milestone and a short bound UTC forward-capture proof. Older PROJECT_AUDIT_2026-10-27.txt described only Seats 1-24 plus incomplete Seat 25 **at its earlier audit point**. The later COUNCIL_IMPLEMENTATION and COUNCIL_CONSOLIDATION notes supersede that older seat count. Never combine old and new audit claims without their dates.

Original Council consolidation: 31 agent implementations; single staged registry/runner path; synthetic research HOLD. Dated archive reports previous regression evidence, not a newly rerun test by this document.

Adaptation milestone 1: additive oracle_memory_schema, oracle_council_cycles, oracle_agent_observations, oracle_forecasts, oracle_forecast_outcomes; hashed/idempotent runs and bounded delayed settlement; production journal initially empty in that checkpoint.

October 27 clock epoch milestone: isolated oracle_utc_feed_schema, oracle_utc_feed_epochs, oracle_utc_feed_captures, oracle_utc_ticks, oracle_utc_bars; ten receipts and short real capture (1,507 first-owned ticks and six M5 bars). Supervisor stopped after the bounded session; future run requires new separate review.

Legacy historical data: existing historical datasets/models cannot be treated as UTC-certified. Historic event/session features require separate clock-aware reconstruction and versioning.

6.2 Milestone/Stage 1-5 engineering history

These labels are the project's engineering delivery program and **must not be conflated** with the original Blueprint's eight conceptual phases (see Section 9).

- **Stage/Phase 1** — **Achieved engineering scope:** 60 synthetic Council research/integration milestones **Completion evidence:** 60/60 reported complete **Operational implication:** Synthetic tests only; not broker-certified
- **Stage/Phase 2** — **Achieved engineering scope:** 20 forward UTC/source/recovery/evaluation engineering checkpoints **Completion evidence:** 20/20 reported complete **Operational implication:** External gates still open
- **Stage/Phase 3** — **Achieved engineering scope:** 24 engineering checkpoints in source lineage, evaluation and acceptance **Completion evidence:** 24/24; Windows 368 passed, 3 skipped, 61 subtests in recorded acceptance; final commit 4b855d3 **Operational implication:** Research acceptance only
- **Stage 4** — **Achieved engineering scope:** 24 forensic/custody/handover checkpoints **Completion evidence:** 24/24; Windows 573 passed, 3 skipped, 61 subtests; final commit 761f60f **Operational implication:** Independent evidence gates not cleared
- **Stage 5** — **Achieved engineering scope:** 24 evidence intake, broker clock, original capture, Council causality, prospective evidence and final handover checkpoints **Completion evidence: 24/24**; Windows **847 passed, 9 skipped, 61 subtests**; final Git commit **fb1c643**, pushed **Operational implication:** Engineering complete; **0/11 external verified**; FINAL=HOLD
- **Proposed Stage 6** — **Achieved engineering scope:** 24 research-first foundation checkpoints, described in Section 10 **Completion evidence: 0/24; not started Operational implication:** No admission or trading approval

Stage 4 six commit batches:

- **S4-01..04** — **Function:** Collector forensics **Git commit:** 00f2618
- **S4-05..08** — **Function:** Broker clock provenance **Git commit:** 9550bc7
- **S4-09..12** — **Function:** Forward receipt recovery **Git commit:** 7ef1d0e
- **S4-13..16** — **Function:** Council input provenance **Git commit:** 375a2fd
- **S4-17..20** — **Function:** Prospective evidence custody **Git commit:** 1e27212
- **S4-21..24** — **Function:** Final forensic handover **Git commit:** 761f60f

Stage 5 six verified/pushed commit batches:

- **S5-01..04** — **Function:** Evidence intake; versioned trust inventory **Git commit:** 1031a5b
- **S5-05..08** — **Function:** Broker identity, UTC origin and clock attestation gaps **Git commit:** 6612b09
- **S5-09..12** — **Function:** Original capture-list hashes and proposed new-epoch prerequisites **Git commit:** 10b9de6
- **S5-13..16** — **Function:** Static Council source family inventory and point-in-time causality **Git commit:** 925eced
- **S5-17..20** — **Function:** Predecision prediction, post-horizon outcome, cost and holdout review **Git commit:** dcccf0b
- **S5-21..24** — **Function:** Five-commit source custody, evidence summary and final Stage 5 handover **Git commit: fb1c643**

Windows test progression: Stage 5 batch 1: 614 passed, 4 skipped; batch 2: 656 passed, 5 skipped; batch 3: 697 passed, 6 skipped; batch 4: 747 passed, 7 skipped; batch 5: 794 passed, 8 skipped; batch 6: **847 passed, 9 skipped**, plus 61 subtests in each recorded full regression. These are cumulative allowlist test results; not 847 new tests and not an external profitability experiment.

Latest Git head evidence: user PowerShell showed fb1c643 at both branch HEAD and origin/feature/utc-research-m01-m08, and no git status --short output. This file has not independently executed a live Git fetch or inspected the private Windows working tree.

7. Proofs, evidence gaps and the difference between tests and truth

7.1 The eleven independent external gates - all still unresolved

- **G01** — **Gate:** INDEPENDENT_BROKER_SERVER_IDENTITY **Genuine proof needed to close it:** Provider-confirmed server/account identity, original receipt and independent trust check **Status:** NOT VERIFIED
- **G02** — **Gate:** INDEPENDENT_BROKER_CLOCK_REFERENCE_AND_OFFSET **Genuine proof needed to close it:** Authenticated UTC reference, broker clock samples, uncertainty and independently reviewed offset **Status:** NOT VERIFIED
- **G03** — **Gate:** INDEPENDENT_ORIGINAL_CAPTURE_RECEIPTS_AND_SOURCE_PAYLOADS **Genuine proof needed to close it:** Original ordered capture lists, immutable broker/source receipts and externally verifiable custody **Status:** NOT VERIFIED
- **G04** — **Gate:** CURRENT_FORWARD_SESSION_WITH_APPROVED_NEW_EPOCH_PLAN **Genuine proof needed to close it:** Separately operator-approved new epoch and current continuous forward-session provenance **Status:** NOT VERIFIED
- **G05** — **Gate:** INDEPENDENT_MARKET_AND_MACRO_SOURCE_PROVENANCE_FOR_13_FAMILIES **Genuine proof needed to close it:** Authentic source origins, timestamped first availability, revisions and complete 13-family records **Status:** NOT VERIFIED
- **G06** — **Gate:** INDEPENDENT_COUNCIL_DERIVED_INPUT_AND_MODEL_PROVENANCE **Genuine proof needed to close it:** Traceable features, model artifacts, approved versions and decision-time dependency closure **Status:** NOT VERIFIED
- **G07** — **Gate:** OPERATOR_APPROVED_RISK_AND_RECOVERY_CONTROLS **Genuine proof needed to close it:** Signed/authenticated risk and recovery procedure with testable veto/kill-switch ownership **Status:** NOT VERIFIED
- **G08** — **Gate:** EXTERNALLY_TIMESTAMPED_PREDECISION_PREDICTIONS **Genuine proof needed to close it:** Trusted append-only independent registration before decisions and forecast horizons **Status:** NOT VERIFIED
- **G09** — **Gate:** INDEPENDENT_POST_HORIZON_OUTCOME_RECEIPTS **Genuine proof needed to close it:** Authentic outcome quote source, first receipt, horizon match and origin integrity **Status:** NOT VERIFIED
- **G10** — **Gate:** BROKER_CONFIRMED_EXECUTION_COSTS_IF_TRADES_EXIST **Genuine proof needed to close it:** Broker-confirmed spread, fees, slippage, financing and fills **if trades exist**; otherwise independently attested no-trade scope **Status:** NOT VERIFIED
- **G11** — **Gate:** INDEPENDENT_PRE_EVALUATION_HOLDOUT_FREEZE **Genuine proof needed to close it:** Third-party/time-stamped dataset/split, model selection and evaluation protocol freeze before outcome evaluation **Status:** NOT VERIFIED

The Stage 5 last real read-only report found **0/11** externally verified, and its Council source-family work found **0/13** independently verified. These numbers are not changed by success of 847 engineering tests. No candidate original capture ordered lists, external prospective evidence index, source vintage index or externally attested new-epoch plan had been supplied to the respective inspectors.

7.2 What has been verified (and what has not)

- **31 manifest seats** — **Correct interpretation:** Source and registry declare implementations and staged execution **Invalid inference to avoid:** 31 profitable or independent signals exist
- **10 stored receipt manifests** — **Correct interpretation:** Ten historical capture manifest rows were inspected **Invalid inference to avoid:** Ten externally authenticated broker receipts
- **1,507 ticks and six bars** — **Correct interpretation:** Snapshot count of first-owned rows in the old isolated UTC feed **Invalid inference to avoid:** Sufficient current/live training or market coverage
- **Original clock convention +7200 s** — **Correct interpretation:** Local observed HFM wall-label mapping **Invalid inference to avoid:** Independent official broker-clock UTC certification
- **13 Council source families** — **Correct interpretation:** Static AgentContext source-facing fields exist **Invalid inference to avoid:** 13 true historical, fully authenticated data feeds
- **Prospective evaluation code** — **Correct interpretation:** Candidate timestamp/hash/lineage contracts exist **Invalid inference to avoid:** Verified registered predictions, post-horizon outcomes or net-profit
- **Saved model evaluations** — **Correct interpretation:** Historical metrics and registries exist **Invalid inference to avoid:** Risk-adjusted trading performance after execution costs
- **FINAL=HOLD** — **Correct interpretation:** Safety/operating denial, not a failed test **Invalid inference to avoid:** A trade recommendation that was secretly executed

8. Unfinished implementation, risks and safeguards

Material gaps requiring future design, evidence, or both:

Clock/domain integrity: current clock code is not byte-exact to the historical epoch; the original broker offset is locally interpreted but not independently authenticated. Preserve all October 27 anchors. Do not disable hash policy to make replay pass.

Restoration of a valid, separately authorized forward session: no current approved new epoch, independently reviewed provider identity, UTC reference or continuous collector plan. No autonomous restart.

Genuine original data custody: first-owned SQL rows cannot reconstruct originally hashed per-capture complete ordered source lists; external source/origin receipts absent.

Causal full-context research: no fully verified 13-family, versioned source intake, Fed production admission, complete current market-state snapshot and 31-agent approved decision loop.

Normalized agent forecasts/calibration: distinct horizons/methodologies cannot be blended as equivalent economic probabilities; correlated agent failures and abstentions must be measured.

Predictive registration and evaluation: no independently time-stamped forecast ledger, post-horizon outcome receipts, held-out freeze, or net-of-costs demonstration.

MiroFish aspiration: Monte Carlo seat exists but no separate validated future-world simulation engine or scenario market model is demonstrated.

Risk operations: Council risk veto exists, but no separately reviewed order sizing/fractional-Kelly, live portfolio reconciliation, risk authority service, kill-switch drill or post-trade ledger.

Execution/dashboard: no authorized paper/live broker order management, completed trading dashboard or validated five-minute continuous Council scheduler; backend/main.py only prints a banner.

Operational security: .env must remain private; direct broker/API/DB read/write scripts should never be run as part of documentation refresh. Maintain a clean source archive excluding venv, .git internals and private logs.

No claim of profitability or live safety is warranted. The original historical test set was already examined; a truly new held-out freeze and independent prospective observations must replace repeated testing on old benchmarks.

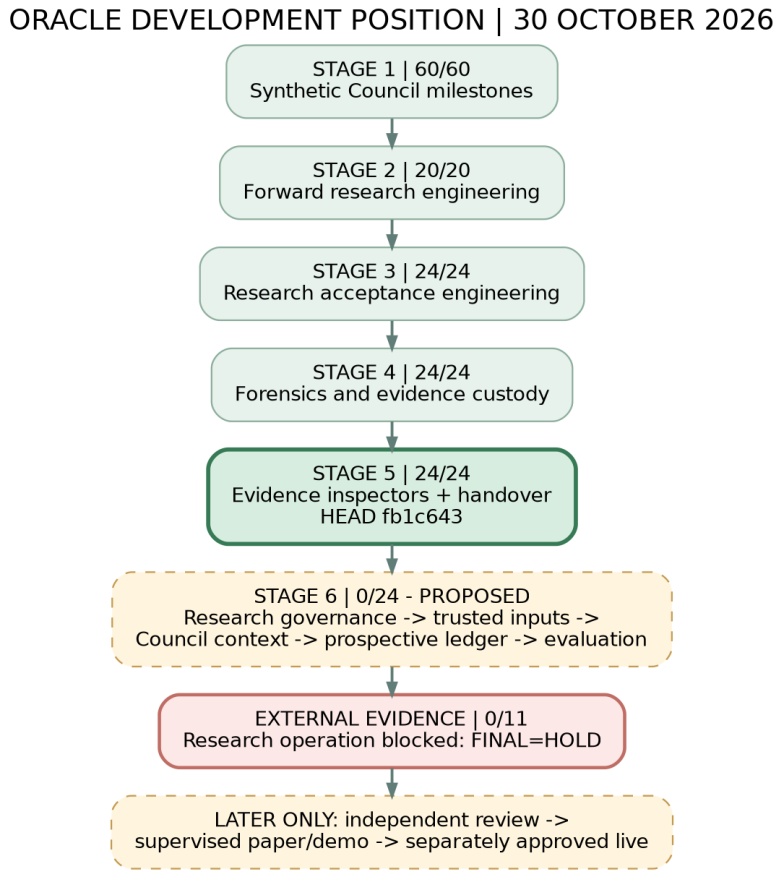
9. Original Blueprint phases versus engineering Stage 6 - do not confuse these

The original v1.1 blueprint specified **eight conceptual phases**: (1) infrastructure, (2) agent framework, (3) intelligence expansion, (4) simulation, (5) consensus, (6) backtesting, (7) paper trading, (8) controlled deployment. The post-October engineering **Stages 1-5** track a *different* milestone system; their numbering does not prove that all eight original blueprint phases are done.

- **1. Infrastructure** — **Source-backed status now:** Many components implemented, historical bounded capture tested **Key missing admission/outcome:** Independent current broker/UTC trust and approved continuous feed
- **2. Agent Framework** — **Source-backed status now:** 31 implementations and canonical staged runner **Key missing admission/outcome:** Fresh trusted inputs; predictive validity
- **3. Intelligence Expansion** — **Source-backed status now:** Quantitative, COT, central-bank and cross-asset modules present **Key missing admission/outcome:** Causal source admissibility and independent model/input lineage
- **4. Simulation Engine** — **Source-backed status now:** Monte Carlo seat implemented **Key missing admission/outcome:** Separate validated MiroFish-style scenario system
- **5. Consensus Engine** — **Source-backed status now:** Governance/chief strategist rule-driven synthesis exists **Key missing admission/outcome:** Reliable empirical weight calibration and correlated-error checks
- **6. Backtesting** — **Source-backed status now:** Historical data/model experiments and metrics exist **Key missing admission/outcome:** Independently held-out, cost-aware walk-forward/trading strategy evaluation
- **7. Paper Trading** — **Source-backed status now:** Not independently approved/operational **Key missing admission/outcome:** Separate controlled demo mode, order lifecycle and risk approval
- **8. Controlled Deployment** — **Source-backed status now:** Not implemented/approved as live trading **Key missing admission/outcome:** Every independent operational/risk gate and explicit separate authorization

The proposed Stage 6 below is a research-first engineering program aligned with Blueprint v1.2. It is not the original blueprint's Phase 6 backtesting and it does not by itself permit demo/live orders.

10. Full planned development program from this checkpoint



10.1 Stage 6 - proposed 24 checkpoints, six four-checkpoint batches

Stage 6 acceptance rule: build safely offline before attempting external source access; every batch includes two reviewed modules/tests where appropriate, focused regressions, full regression, real **read-only** assessment, scoped Git commit/push and an updated atlas. Any task needing third-party proof stays BLOCKED rather than being marked complete by synthetic test success. The following is a *proposed design plan*, not delivered code.

- **P6-01** — **Planned deliverable:** Versioned research-policy contract, mode authority and immutable historical anchor register **Acceptance / gate:** Explicit research-only defaults; unchanged legacy epoch
- **P6-02** — **Planned deliverable:** Evidence source taxonomy and trust levels (local, externally attested, quarantined) **Acceptance / gate:** No source promoted by hash alone
- **P6-03** — **Planned deliverable:** Research acceptance matrix mapping all 11 gates to custody, owner and independent proof **Acceptance / gate:** Machine- and human-readable blockers; all currently open
- **P6-04** — **Planned deliverable:** Deterministic research plan/handover report plus safety regression **Acceptance / gate:** Fully offline HOLD; docs updated and committed
- **P6-05** — **Planned deliverable:** Unified market tick/M5 receipt and revision envelope (design/validation only) **Acceptance / gate:** Provider, raw clock, availability and uncertainty captured
- **P6-06** — **Planned deliverable:** ECB/COT/Fed/cross-asset versioned source document and revision contract **Acceptance / gate:** Future actuals and backdated releases refused

- **P6-07** — **Planned deliverable:** Source continuity, gap, clock and stale-evidence quality scoring **Acceptance / gate:** No silent fill-forward or fabricated availability
- **P6-08** — **Planned deliverable:** Independent custody import adapter specification and offline verification **Acceptance / gate:** No MT5 restart/network writes; unverified input quarantined
- **P6-09** — **Planned deliverable:** Reproducible point-in-time market snapshot schema for Council **Acceptance / gate:** Source hashes, feature versions, availability cutoffs frozen
- **P6-10** — **Planned deliverable:** 31-seat input-entitlement and readiness map with contextual versus directional types **Acceptance / gate:** Missing/blocked data cause abstention/HOLD
- **P6-11** — **Planned deliverable:** Deterministic Council replay/context comparison without live dispatch **Acceptance / gate:** Stable hashes; independent risk veto retained
- **P6-12** — **Planned deliverable:** Council context audit and gate coverage against all 13 families **Acceptance / gate:** External source availability independently documented or blocked
- **P6-13** — **Planned deliverable:** External predecision prediction registration contract **Acceptance / gate:** Claim cannot become authoritative without trusted receipt
- **P6-14** — **Planned deliverable:** Outcome-horizon linkage, quote-source and no-look-ahead protocol **Acceptance / gate:** First receipt, horizon and source authenticity checked
- **P6-15** — **Planned deliverable:** Cost/no-trade scope, fills and transaction-cost evidence model **Acceptance / gate:** Never invent fills; no verified net result without costs
- **P6-16** — **Planned deliverable:** Append-only prospective ledger reconciliation and quarantine report **Acceptance / gate:** Idempotent identity, mutations blocked, HOLD
- **P6-17** — **Planned deliverable:** Simple predeclared benchmarks: no-trade, naive direction, rule-only **Acceptance / gate:** Explicit target, horizon, class balance, baseline contract
- **P6-18** — **Planned deliverable:** Council disagreement, abstention, governance veto and correlation analysis **Acceptance / gate:** Prospective records only; no ex-post invented votes
- **P6-19** — **Planned deliverable:** Calibration, drift and regime-conditioned performance evaluation plan **Acceptance / gate:** New held-out freeze and minimum sample/uncertainty rules
- **P6-20** — **Planned deliverable:** Net-of-costs and risk-adjusted scorecard with sensitivity analysis **Acceptance / gate:** Costs, slippage stress, confidence intervals, no profit assertion
- **P6-21** — **Planned deliverable:** Stage 6 evidence and engineering ancestry audit **Acceptance / gate:** Commit and report hashes locally validated
- **P6-22** — **Planned deliverable:** All 11 external gates reassessed by authenticated evidence **Acceptance / gate:** Cannot clear unproven gate, even if engineering complete
- **P6-23** — **Planned deliverable:** Research readiness and operational safety handover **Acceptance / gate:** No automatic Council/collector/trading activation
- **P6-24** — **Planned deliverable:** Frozen Stage 6 transition record and next-phase decision **Acceptance / gate:** Explicit engineering versus operational verdict

Suggested implementation cadence: six batches of four; each batch first creates offline source and tests, then runs read-only checks on the real repository and requires a distinct Git commit. Exact modules, metrics and sample sizes remain to be specified and tested. Do not pre-mark any P6 checkpoint validated or committed.

10.2 After Stage 6 - conditional research-to-execution roadmap

- **R1. Independent acquisition and source custody** — **Purpose and logical order:** Obtain broker/UTC confirmations, true original receipts, trustworthy versions of all macro data, and a separately approved epoch plan **Required evidence before advancing:** G01-G06, with reviewed current evidence and no historical rebind
- **R2. Controlled offline/forward research** — **Purpose and logical order:** Shadow research cycles after separate authority; immutable causal context, comprehensive logging, independently registered predictions and matured outcomes **Required evidence before advancing:** G07-G09 plus current-session integrity and risk stop conditions
- **R3. Model comparison and evidence-backed policy** — **Purpose and logical order:** Head-to-head Council vs fixed baselines; decision-veto value, calibration, correlated agent error and costs **Required evidence before advancing:** G10-G11, sufficient prospective outcomes, explicit uncertainty thresholds
- **R4. Supervised paper/demo operations** — **Purpose and logical order:** Isolated order simulator/demo account, reconciliation, risk authority, kill-switch, monitored recovery **Required evidence before advancing:** Distinct operator authorization; never inferred from Git status
- **R5. Cautious live feasibility assessment** — **Purpose and logical order:** Independent security, broker execution and market-risk review; staged caps and rollback design **Required evidence before advancing:** New authorization process; cannot be approved by this documentation
- **R6. Monitored and controlled production, if ever authorized** — **Purpose and logical order:** Separate execution service, observability, alerts, revocation and periodic independent review **Required evidence before advancing:** Demonstrated sustained evidence of net benefit and safe operation

This full plan retains original Blueprint ideas (MiroFish simulations, adaptive reliability, fractional-Kelly sizing, dashboard) as future research/execution projects, not achievements. No timetable or profitability target is promised.

11. Safe operating and resumption instructions for the next developer/chat

Start every new conversation by reading: this atlas, Blueprint v1.2, docs/ historical evidence, the currently checked-out Git commit and its status, followed by the specific files for the next checkpoint. Always establish whether an observation comes from the dated ZIP, the local real checkout, Windows test logs, or conceptual plan.

Expected Windows repository path: C:\Users\user\EURUSD_ORACLE; expected branch feature/utc-research-m01-m08; latest reported commit fb1c643. These are context, not grounds to delete work or force-reset.

Safe initial verification (no broker or DB side effects):

```
git status --short
git branch --show-current
git log -1 --oneline
git rev-parse HEAD
git rev-parse origin/feature/utc-research-m01-m08
python -m py_compile backend/runtime/stage5_final_research_handover.py
```

The last Windows allowlist regression passed 847, skipped 9 and reported 61 additional subtests; do not claim this test result is a new run. To inspect code, use file reads and safe AST parsing, not main.py, MT5 recorder/ingestors, model fitting commands, supervisor startup, or a Council CLI.

Never automatically: start/restart an MT5 session or collector; access credentials; rebind/renew the historical policy; append to production SQL; train/promote a model; unfreeze the previous test set; dispatch the Council; settle true outcomes; submit orders; change risk permissions. The program's standing operational state is **FINAL=HOLD** until independent evidence gates and distinct operator decisions justify any other permitted mode.

Mandatory documentation discipline from Stage 6 onward

For **every** batch/checkpoint, append a dated record with: (1) checkpoint IDs and objective; (2) exact created/modified paths; (3) file-level purpose, interfaces and dependencies; (4) safety/no-side-effect boundary; (5) fixed historical anchors affected/not affected; (6) tests and skipped reasons; (7) real read-only inspection outputs and SHA-256; (8) actual Git commit and push verification; (9) unresolved blockers/external proof; (10) changed architecture and next steps. Use three distinct tags: ENGINEERING_COMPLETE, LOCAL_EVIDENCE_ONLY, INDEPENDENTLY_VERIFIED. Do not substitute one for another.

Suggested ledger entry template:

Stage/Checkpoint: [ID] - [UTC date]
Source basis: *[current live checkout / dated ZIP / user terminal log]*
Changed files: *[exact paths]*; Ownership/interfaces: *[...]*
Design and safety boundary: *[...]*; No-side-effects checks: *[...]*
Focused tests: *[N passed / N skipped]*; Full regression: *[...]*
Real read-only inspection: *[command / as-of / result / report SHA-256]*
External gates: *[independent evidence, issuer, receipt / still missing]*
Git commit: *[full ID]*; Remote tracking: *[confirmed / not checked]*
Disposition: *[engineering complete / blocked / pending]*
Next action: *[...]*

12. Directly inspected October 27 source inventory - file-by-file

The following entries cover **every one of the 316 first-party backend .py files** in the supplied archive (including test modules and intentionally empty stubs), organized by actual folder. The inventory was generated by reading the source bytes and parsing AST, with file-specific notes grounded in module docstrings, top-level classes/functions, imports and selected code-path reviews. Each file includes its source line count and an SHA-256 prefix to aid comparison. It is a navigation/audit catalog, **not a guarantee that every named function is production-ready**.

12.1. backend/agents/ - 77 Python files (52 non-test; 25 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

agent_registry.py (419 lines; SHA c7b0f4901957) - Registry enforces canonical agent identities/specification, registration and retrieval for Council execution. Main symbols: AgentRegistry.

analysis_helpers.py (48 lines; SHA e175066da812) - Pure calculations shared by remaining Council seats; no data acquisition. Main symbols: validated_analysis, missing_primary.

base_agent.py (298 lines; SHA ed6edaefba7f) - Abstract per-agent execution lifecycle and opinion validation shared by Council implementations. Main symbols: BaseAgent.

context_factory.py (461 lines; SHA 0485bc2b3995) - Constructs validated, causal AgentContext from market state, macro stores and bounded supplemental research inputs. Main symbols: AgentContextFactory.

contracts.py (969 lines; SHA 9fdb0986ddc9) - Pydantic/typed data models for agent identity, stages, context, opinion and Council data provenance. Main symbols: AgentCategory, AgentStage.

council_analysis_context.py (25 lines; SHA 86fc8e22f8ac) - Compatibility research view of the canonical AgentContext. Main symbols: CouncilAnalysisContext.

council_inputs.py (149 lines; SHA a45bafff2a71) - Shared typed Council inputs, independent of registry and execution mode. Main symbols: FrozenInput, CrossAssetObservation.

council_manifest.py (528 lines; SHA 1e495c02f8f3) - Authoritative immutable 31-seat catalogue, ordering, categories, stages, purposes and required inputs.

council_runner.py (225 lines; SHA ae02edb53aa6) - Executes canonical primary/governance/synthesis stages with explicit errors/blocks, deterministic order and no broker authorization. Main symbols: AgentRunStatus, AgentExecutionResult.

dealer_behaviour_features.py (759 lines; SHA afcc1c5e8394) - Constructs bounded dealer-behaviour proxy features from market microstructure observations; no actual dealer book is observed. Main symbols: DealerBehaviourFeatures, DealerBehaviourFeatureEngine.

execution_policy.py (76 lines; SHA 5cb2ca864844) - Central dispatch eligibility. Registration never grants source admission. Main symbols: AgentEligibility.

fed_context_adapter.py (282 lines; SHA 655056e07c67) - F4.5.3: Quarantined Fed input preflight for an operational context. Main symbols: FedContextAdapterError, FedContextQuarantineReceipt.

fed_input_readiness.py (141 lines; SHA 511dab8b554c) - F4.5.2a: Read-only Fed Council input readiness diagnostics. Main symbols: FedRateInputStatus, FedInputNotReady.

fed_live_source_admission.py (260 lines; SHA af6672769310) - F5.2: current HTTPS Fed source observation into a one-use live preview. Main symbols: FedLiveSourceAdmissionError, FedLiveCaptureReceipt.

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

anomaly_detection_agent.py (295 lines; SHA d15f44ac9a5b) - Detect abnormal market behaviour that may invalidate ordinary models. Source requirements: candles, market_state. Main symbols: AnomalyDetectionAgent.

chief_strategist_agent.py (89 lines; SHA a73cc29582a4) - Synthesize analytical and governance opinions into the final Council view. Source requirements: primary_agent_opinions, governance_opinions, market_state. Main symbols: ChiefStrategistAgent.

cot_positioning_agent.py (265 lines; SHA 11e35fc20606) - Evaluate institutional positioning from validated positioning data. Source requirements: cot_data, market_state. Main symbols: COTPositioningAgent.

dealer_behaviour_agent.py (256 lines; SHA fbc660ea4904) - Evaluate dealer-style inventory and microstructure behaviour. Source requirements: tick_features, market_state. Main symbols: DealerBehaviourAgent.

devils_advocate_agent.py (45 lines; SHA 865b5b10507b) - Challenge dominant Council reasoning and search for contradictory evidence. Source requirements: primary_agent_opinions, market_state. Main symbols: DevilsAdvocateAgent.

ecb_agent.py (724 lines; SHA d0390ff2bf46) - Evaluate ECB policy and euro-side monetary conditions. Source requirements: ecb_data, economic_calendar. Main symbols: ECBAgent.

economic_calendar_agent.py (43 lines; SHA 2cca3dee6abd) - Evaluate scheduled macroeconomic event risk and surprise information. Source requirements: economic_calendar. Main symbols: EconomicCalendarAgent.

fed_agent.py (132 lines; SHA 42c29e43f52d) - Evaluate Federal Reserve policy and US dollar monetary conditions. Source requirements: fed_data, economic_calendar. Main symbols: FedAgent.

fibonacci_agent.py (276 lines; SHA 6818ef94a13c) - Evaluate retracement and extension relationships around confirmed swings. Source requirements: candles, market_structure. Main symbols: FibonacciAgent.

hidden_markov_model_agent.py (651 lines; SHA ec67c6bc4dbd) - Estimate latent market regime and regime-transition probabilities. Source requirements: candles, model_artifact. Main symbols: HiddenMarkovModelAgent.

liquidity_hunter_agent.py (368 lines; SHA fb0d2d12f13b) - Evaluate liquidity concentration, sweeps and likely liquidity targets. Source requirements: liquidity_state, market_structure, tick_features. Main symbols: LiquidityHunterAgent.

london_session_agent.py (17 lines; SHA 7803f6e9fc48) - Evaluate EUR/USD behaviour in the London trading session. Source requirements: session_state, candles. Main symbols: LondonSessionAgent.

market_structure_agent.py (150 lines; SHA dedb4057aa6b) - Evaluate confirmed swing structure and structural transitions. Source requirements: market_state, candles. Main symbols: MarketStructureAgent.

mean_reversion_agent.py (191 lines; SHA 2705eeca43fb) - Estimate whether price displacement is statistically likely to revert. Source requirements: candles, market_state. Main symbols: MeanReversionAgent.

ml_tree_ensemble_agent.py (646 lines; SHA 157788db3469) - Generate predictions from validated tree-based machine-learning models. Source requirements: feature_vector, model_artifact. Main symbols: MLTreeEnsembleAgent.

momentum_agent.py (106 lines; SHA 2ee41998bab3) - Evaluate current directional force and momentum acceleration. Source requirements: market_state, candles. Main symbols: MomentumAgent.

monte_carlo_agent.py (267 lines; SHA 21f60e13bdb2) - Estimate directional outcome distributions through simulation. Source requirements: candles, market_state. Main symbols: MonteCarloAgent.

neural_network_agent.py (752 lines; SHA e808b81e5d70) - Generate nonlinear predictions from validated neural models. Source requirements: feature_vector, model_artifact. Main symbols: NeuralNetworkAgent.

new_york_session_agent.py (17 lines; SHA 36f6d7bdbdb0) - Evaluate EUR/USD behaviour in the New York trading session. Source requirements: session_state, candles. Main symbols: NewYorkSessionAgent.

order_flow_agent.py (225 lines; SHA 093cd8bdd55d) - Evaluate tick-level directional pressure and microstructure imbalance. Source requirements: tick_features, liquidity_state. Main symbols: OrderFlowAgent.

pattern_recognition_agent.py (238 lines; SHA 1f6e291da057) - Detect recurring candle and geometric price patterns. Source requirements: candles. Main symbols: PatternRecognitionAgent.

price_action_agent.py (270 lines; SHA 75509c9cd8fd) - Evaluate recent candle behaviour, rejection and directional pressure. Source requirements: candles. Main symbols: PriceActionAgent.

rate_differential_agent.py (43 lines; SHA 2db625e067d5) - Evaluate relative euro and dollar interest-rate conditions. Source requirements: rates_data. Main symbols: RateDifferentialAgent.

risk_officer_agent.py (81 lines; SHA a1fd7fafdbe2) - Evaluate whether current conditions justify taking market risk. Source requirements: primary_agent_opinions, market_state, risk_state. Main symbols: RiskOfficerAgent.

risk_sentiment_agent.py (43 lines; SHA ce5e9d845cc6) - Evaluate broader cross-asset risk-on and risk-off conditions. Source requirements: cross_asset_data, market_state. Main symbols: RiskSentimentAgent.

session_behavior_base.py (509 lines; SHA f11bc172e084) - Shared London/New York session behavior implementation used by both session-specific agent seats. Main symbols: SessionBehaviorAgent.

smart_money_agent.py (381 lines; SHA 7f5bbaba2b1f) - Evaluate displacement, liquidity behaviour and institutional-style structure. Source requirements: candles, market_structure, tick_features. Main symbols: SmartMoneyAgent.

statistical_probability_agent.py (415 lines; SHA 6d660ecea066) - Estimate direction from empirical conditional probabilities. Source requirements: candles, market_state. Main symbols: StatisticalProbabilityAgent.

support_resistance_agent.py (307 lines; SHA a5cd7376f880) - Evaluate price relative to meaningful support and resistance zones. Source requirements: candles. Main symbols: SupportResistanceAgent.

time_series_agent.py (300 lines; SHA 09608ede692b) - Evaluate temporal dependence and short-horizon directional forecasts. Source requirements: candles. Main symbols: TimeSeriesAgent.

trend_master_agent.py (109 lines; SHA 0897913b9fed) - Evaluate persistent directional trend conditions. Source requirements: market_state, candles. Main symbols: TrendMasterAgent.

volatility_agent.py (100 lines; SHA bda888c8f489) - Evaluate the current volatility regime and volatility-adjusted opportunity. Source requirements: market_state, candles. Main symbols: VolatilityAgent.

microstructure_features.py (398 lines; SHA d015b640d060) - Converts quote/tick records into microstructure features for liquidity/order-flow/dealer seats. Main symbols: MicrostructureFeatures, MicrostructureFeatureEngine.

register_core_agents.py (143 lines; SHA 32f1ca516639) - Authoritative implementation catalogue for all 31 Council seats. Main symbols: register_core_agents.

research_council.py (30 lines; SHA 3e0ce1a59ad6) - Compatibility research adapters around the canonical catalogue and runner. Main symbols: ResearchCouncilRunner.

run_research_council.py (129 lines; SHA 0df5d3cccad1) - Explicit, bounded JSON entry point for isolated full-Council research. Main symbols: reject_duplicates, reject_nonfinite.

Test modules

test_agent_registry.py (312 lines; SHA be8428678c04) - Procedural regression/fixture for agent registry; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_base_agent_framework.py (396 lines; SHA becd5bc61d56) - Procedural regression/fixture for base agent framework; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_canonical_council.py (266 lines; SHA a71bd09d43ca) - Tests canonical council code/contracts; 22 pytest-style test functions/methods, including one catalogue matches all manifest seats in order; registration is distinct from mode eligibility; default registration includes all 31 real implementations Archived source only; current run status is not established by file presence.

test_context_factory_macro.py (354 lines; SHA 26e101b2c4b1) - Procedural regression/fixture for context factory macro; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_core_market_agents.py (640 lines; SHA 12a0078be90b) - Procedural regression/fixture for core market agents; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_core_market_agents_historical.py (873 lines; SHA 1e5ef308a0ec) - Procedural regression/fixture for core market agents historical; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cot_positioning.py (587 lines; SHA d0a19b476ccc) - Procedural regression/fixture for cot positioning; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_council_runner.py (677 lines; SHA fb6057c0ed7a) - Procedural regression/fixture for council runner; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_dealer_behaviour.py (571 lines; SHA 7943080dfe38) - Procedural regression/fixture for dealer behaviour; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_dealer_behaviour_features.py (625 lines; SHA 39a9764413d9) - Procedural regression/fixture for dealer behaviour features; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_ecb.py (925 lines; SHA c2ba0eab5f4f) - Procedural regression/fixture for ecb; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_fed_agent.py (237 lines; SHA f1d03b38d548) - Tests fed agent code/contracts; 16 pytest-style test functions/methods, including 01 manifest identity and contextual type; 02 missing fed input rejected; 03 single decision contextual and zero confidence Archived source only; current run status is not established by file presence.

test_fed_context_adapter.py (354 lines; SHA f40f56fb9130) - Tests fed context adapter code/contracts; 21 pytest-style test functions/methods, including 01 empty input stays absent and blocked; 02 complete pair receipt is only diagnostic; 03 base context is not modified Archived source only; current run status is not established by file presence.

test_fed_context_contract.py (364 lines; SHA 8d370914cc57) - Tests fed context contract code/contracts; 17 pytest-style test functions/methods, including 01 optional fed data defaults to empty; 02 complete fed pair accepted; 03 announced before effective date is allowed Archived source only; current run status is not established by file presence.

test_fed_council_readiness.py (280 lines; SHA d42b6211fd12) - Tests fed council readiness code/contracts; 14 pytest-style test functions/methods, including 01 manifest has 31 unique ordered seats; 02 fed seat manifest contract; 03 manifest fields exist in context contract Archived source only; current run status is not established by file presence.

test_fed_input_readiness.py (320 lines; SHA b479d0c489f6) - Tests fed input readiness code/contracts; 18 pytest-style test functions/methods, including 01 absent rates are reported; 02 f1 valid pair is diagnostic only; 03 two decisions are counted Archived source only; current run status is not established by file presence.

test_fed_live_candidate_ingress.py (276 lines; SHA 65b59c9a26f6) - Tests fed live candidate ingress code/contracts; 19 pytest-style test functions/methods, including 01 no fed rows return no candidates; 02 complete pair passes as diagnostic candidates; 03 future ingestion cannot leak even if query misbehaves Archived source only; current run status is not established by file presence.

test_fed_live_source_admission.py (219 lines; SHA 29279f556284) - Tests fed live source admission code/contracts; 22 pytest-style test functions/methods, including 01 real client type uses mocked original bundle; 02 observed after capture live preview populates fed data; 03 only genuinely paired data can reach preview Archived source only; current run status is not established by file presence.

test_full_research_council.py (65 lines; SHA add49135422f) - Tests full research council code/contracts; 1 pytest-style test functions/methods, including all real seats on synthetic causal inputs Archived source only; current run status is not established by file presence.

test_hidden_markov_model.py (653 lines; SHA c9fc21f8a4b1) - Procedural regression/fixture for hidden markov model; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_microstructure_features.py (270 lines; SHA 2dca73d32e34) - Procedural regression/fixture for microstructure features; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_ml_tree_ensemble.py (717 lines; SHA f470e16a06b1) - Procedural regression/fixture for ml tree ensemble; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_neural_network.py (1113 lines; SHA 53da4580a679) - Procedural regression/fixture for neural network; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_remaining_council_seats.py (486 lines; SHA 56ca295f73d7) - Tests remaining council seats code/contracts; 52 pytest-style test functions/methods, including base context is preserved; cannot replace base context through supplements; analysis mode is research only Archived source only; current run status is not established by file presence.

test_research_council_input.py (54 lines; SHA 2a0a920f85f6) - Tests research council input code/contracts; 6 pytest-style test functions/methods, including valid context round trip; duplicate keys rejected at any depth; nonfinite json constants rejected Archived source only; current run status is not established by file presence.

12.2. backend/candle_engine/ - 12 Python files (11 non-test; 1 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

broker_candle_storage.py (89 lines; SHA a10fb6fd1ab9) - Persists broker-origin candle observations in the broker-candle schema. Main symbols: BrokerCandleStorage.

candle_builder.py (60 lines; SHA db61816ab792) - Builds candle bars from incoming market prices using candle boundaries. Main symbols: CandleBuilder.

candle_model.py (25 lines; SHA 3b1e3dce9341) - Defines the candle value record for the legacy candle engine. Main symbols: Candle.

candle_scheduler.py (0 lines; SHA e3b0c44298fc) - Intentionally empty placeholder in archived source; no substantive implementation in this file.

candle_storage.py (110 lines; SHA d2089f62c4ad) - Stores calculated candles and handles duplicate/update behavior. Main symbols: CandleStorage.

live_candle_engine.py (163 lines; SHA 2ea061e51fbc) - Coordinates real-time candle assembly and storage; an active runtime worker, not an audit command. Main symbols: LiveCandleEngine.

mt5_bar_ingestor.py (240 lines; SHA 98576b4eb5ca) - Pulls and validates broker M5 bars for database ingestion, under timestamp/availability constraints. Main symbols: MT5BarIngestor.

run_live_candle.py (43 lines; SHA 9ee05f15fbff) - Executable CLI to start the legacy live candle pipeline; not authorized by this handover. Main symbols: main.

run_mt5_bar_ingestor.py (219 lines; SHA f307c9a07daf) - CLI/process entry for a bounded/continuous MT5 bar worker with signal shutdown. Main symbols: _request_shutdown, _install_signal_handlers.

timeframe.py (37 lines; SHA c1bef1baf0a7) - Helper for timeframe-aligned boundaries and preceding candle windows. Main symbols: floor_time, previous_window.

Test modules

test_candle.py (48 lines; SHA b726e77eeb4f) - Procedural regression/fixture for candle; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

12.3. backend/config/ - 3 Python files (3 non-test; 0 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

credentials.py (20 lines; SHA b5e927874774) - Reads runtime authentication/configuration settings; never reproduce credential values.

settings.py (9 lines; SHA 9e71060352c4) - Defines the application name, version and environment/configuration defaults.

Test modules

12.4. backend/data_engine/ - 22 Python files (17 non-test; 5 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

base_provider.py (24 lines; SHA dc383262b654) - Abstract contract for market price/tick/candle providers. Main symbols: MarketDataProvider.

batch_tick_storage.py (139 lines; SHA 279c8ec1c61e) - Deduplicated/batched tick persistence used by the legacy ingest pipeline. Main symbols: BatchTickStorage.

candle_builder.py (0 lines; SHA e3b0c44298fc) - Intentionally empty placeholder in archived source; no substantive implementation in this file.

clock_policy.py (216 lines; SHA 7424e09b83bb) - Explicit HFM broker-wall-clock conversion, bound to one observed server. Main symbols: ClockPolicyError, ClockPolicy.

forward_store.py (235 lines; SHA ef90665a9c04) - Separate, forward-only UTC feed epochs. Never writes legacy market tables. Main symbols: ForwardFeedStore.

historical_provider.py (0 lines; SHA e3b0c44298fc) - Intentionally empty placeholder in archived source; no substantive implementation in this file.

mt5_provider.py (356 lines; SHA 2f9078d2b243) - MT5 market-data adapter with explicit transport and UTC-clock quarantine. Main symbols: MT5TimestampIntegrityError, MT5Provider.

mt5_tick_ingestor.py (475 lines; SHA 49c6bce795eb) - MT5 tick-range ingestion and checkpointing into legacy tick tables; may connect/write when run. Main symbols: MT5TickIngestor.

run_tick_collector.py (15 lines; SHA 96ea5feabc89) - Executable command to start the tick collector worker. Main symbols: main.

run_tick_ingestor.py (219 lines; SHA 8d65060d03b8) - Long-running/bounded MT5 tick ingestion CLI and graceful shutdown controller. Main symbols: _request_shutdown, _install_signal_handlers.

storage.py (51 lines; SHA 18d80d5cada0) - Basic tick persistence adapter for the legacy data pipeline. Main symbols: TickStorage.

tick_collector.py (123 lines; SHA 21c0959d6b33) - Coordinates polling of market provider and tick validation/storage. Main symbols: TickCollector.

tick_identity.py (31 lines; SHA 2fba7ba7ac39) - Builds stable fingerprints to deduplicate identical market ticks. Main symbols: create_tick_fingerprint.

tick_model.py (25 lines; SHA f7777d81f0cd) - Defines the timestamped MarketTick value record. Main symbols: MarketTick.

tick_processor.py (0 lines; SHA e3b0c44298fc) - Intentionally empty placeholder in archived source; no substantive implementation in this file.

validator.py (28 lines; SHA 2ee2f6fcacab) - Checks whether incoming market tick values meet basic schema/quality expectations. Main symbols: TickValidator.

Test modules

test_clock_policy.py (162 lines; SHA d4a769fc9b3c) - Tests clock policy code/contracts; 18 pytest-style test functions/methods, including winter summer and year boundaries round trip exact milliseconds; ambiguous nonexistent and undocumented transition labels fail; conversion is independent of host current timezone Archived source only; current run status is not established by file presence.

test_forward_store.py (138 lines; SHA 1fe8cee22f20) - Tests forward store code/contracts; 13 pytest-style test functions/methods, including normalized and original time are both durable; duplicate retry retains first receipt and no duplicate ticks; future receipt or open bar cannot be persisted Archived source only; current run status is not established by file presence.

test_mt5.py (45 lines; SHA e1574d71ea5b) - Procedural regression/fixture for mt5; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_mt5_history.py (165 lines; SHA d54b17baf315) - Procedural regression/fixture for mt5 history; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_mt5_timestamp_boundary.py (362 lines; SHA 2d90ac9fcc22) - Tests mt5 timestamp boundary code/contracts; 14 pytest-style test functions/methods, including 01 current utc quote is accepted unchanged; 02 three hour future quote is blocked; 03 stale quote is blocked Archived source only; current run status is not established by file presence.

12.5. backend/data_quality/ - 4 Python files (4 non-test; 0 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

event_storage.py (49 lines; SHA 5ee647e35143) - Stores typed data-quality incidents for diagnostic review. Main symbols: DataQualityEventStorage.

quality_engine.py (695 lines; SHA af0d5a47146d) - Runs data-quality rules on tick/candle consistency, coverage, freshness and anomalies. Main symbols: DataQualityEngine.

run_quality_check.py (103 lines; SHA ec26d1200440) - CLI for data-quality inspection; distinguish its behavior from a source attestation. Main symbols: main.

Test modules

12.6. backend/database/ - 6 Python files (5 non-test; 1 tests)

Application and supporting modules

connection.py (20 lines; SHA f8e982834a3c) - Creates SQLAlchemy engine/session bindings from configured database connection settings.

init_db.py (20 lines; SHA c6be3354feb0) - Explicit database-table initialization entry; never invoke casually on production data. Main symbols: initialize.

models.py (774 lines; SHA 7360b2ee222d) - SQLAlchemy ORM schema for legacy ticks, candles, broker candles, COT, macro, model and other stored records. Main symbols: Tick, Candle.

optimize_training_indexes.py (51 lines; SHA 71ce8c0fb772) - Adds/indexes query paths used by historical training and dataset audit; mutating operation. Main symbols: optimize_training_indexes.

utc_type.py (24 lines; SHA 196b47ccedf0) - Shared aware-UTC SQL type with an explicit SQLite round-trip convention. Main symbols: UTCInstant.

Test modules

test_connection.py (38 lines; SHA 750aad53f479) - Procedural regression/fixture for connection; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

12.7. backend/institutional/ - 20 Python files (10 non-test; 10 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

cftc_cot_backfill.py (367 lines; SHA a2f8ac30e898) - Plans/acquires historical CFTC TFF COT records and saves validated backfill batches. Main symbols: COTBackfillResult, CFTCCOTBackfill.

cftc_release_calendar.py (260 lines; SHA 43d304a07b0e) - Models publication dates and delays for causal CFTC COT availability. Main symbols: CFTCReleaseCalendar.

cftc_tff_client.py (920 lines; SHA 6298d3462238) - Client/parsers for CFTC Traders in Financial Futures futures-only releases. Main symbols: CFTCTFFFuturesOnlyClient.

cot_development_dataset.py (1015 lines; SHA 59a9af4b6c0e) - Builds COT-event research dataset and chronological splits under publication lag rules. Main symbols: DevelopmentSplitWindow, COTDevelopmentEvent.

cot_feature_builder.py (666 lines; SHA bd2cf2f2e000) - Transforms historic COT positions into bounded positioning vectors/features. Main symbols: COTFeatureVector, COTFeatureBuilder.

cot_repository.py (377 lines; SHA 20c03b1607b3) - Queries COT snapshots available as-of a research decision, with causal filtering. Main symbols: COTPositioningRepository.

cot_research_protocol.py (225 lines; SHA 9428d82bb912) - Defines and validates COT research provenance and release-time policy.

cot_schemas.py (200 lines; SHA fb8748daac3e) - Typed COT snapshot schemas and constrained fields. Main symbols: COTPositioningSnapshot.

cot_storage.py (249 lines; SHA 0f95a06b3ff2) - Persists validated COT data and records storage outcomes. Main symbols: COTStorageResult, COTPositioningStorage.

Test modules

test_cftc_cot_backfill.py (491 lines; SHA 2576fd4cf00d) - Procedural regression/fixture for cftc cot backfill; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cftc_release_calendar.py (246 lines; SHA 35ee873f0ece) - Procedural regression/fixture for cftc release calendar; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cftc_tff_client.py (800 lines; SHA 4ba9472f1504) - Procedural regression/fixture for cftc tff client; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cot_context.py (644 lines; SHA 99b6517ec9fd) - Procedural regression/fixture for cot context; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cot_development_dataset.py (544 lines; SHA 22ef8fda281a) - Procedural regression/fixture for cot development dataset; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cot_feature_builder.py (837 lines; SHA 626e93cc7d37) - Procedural regression/fixture for cot feature builder; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cot_repository.py (769 lines; SHA 8a8f5a7fdd80) - Procedural regression/fixture for cot repository; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cot_research_protocol.py (188 lines; SHA 7c8b9df06bf8) - Procedural regression/fixture for cot research protocol; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cot_schemas.py (300 lines; SHA 41c447eb944d) - Procedural regression/fixture for cot schemas; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_cot_storage.py (933 lines; SHA ea48a7b8b6b7) - Procedural regression/fixture for cot storage; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

12.8. backend/learning/ - 9 Python files (7 non-test; 2 tests)

Application and supporting modules

__init__.py (1 lines; SHA e39e3826eace) - Package initializer and package-level documentation.

contracts.py (96 lines; SHA 862697f98a54) - Explicit targets and provenance for append-only Council memory. Main symbols: MemoryInput, RunProvenance.

database.py (14 lines; SHA 622755f4c8db) - Explicit journal destination selection, without connection-string output. Main symbols: memory_engine.

memory.py (297 lines; SHA 779e0ed318e6) - Atomic, idempotent Council records and delayed, append-only outcomes. Main symbols: MemoryConflict, MemoryIntegrityError.

schema.py (51 lines; SHA 261c7e259cc2) - Isolated additive tables; never calls the legacy Base.metadata.create_all.

service.py (56 lines; SHA 18248eaaee88) - Record the canonical runner; no alternate agent catalogue or orchestration. Main symbols: RecordedCouncilRun, CouncilMemoryService.

settle_outcomes.py (70 lines; SHA 33e7146e233e) - Score explicitly supplied mature quotes. Does not fetch data, fit or trade. Main symbols: SettlementRequest.

Test modules

test_memory.py (344 lines; SHA 49368211ecc2) - Tests memory code/contracts; 34 pytest-style test functions/methods, including cycle stores all seats inputs models policy and abstention; duplicate cycle ignores

durations and keeps original record time; changed policy or code cannot overwrite cycle Archived source only; current run status is not established by file presence.

test_settlement_input.py (35 lines; SHA 4bcd63949c1a) - Tests settlement input code/contracts; 2 pytest-style test functions/methods, including valid request preserves quote origin and identity; duplicate keys nonfinite oversize and untyped batches fail Archived source only; current run status is not established by file presence.

12.9. backend/macro/ - 78 Python files (38 non-test; 40 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

capture_cross_asset.py (30 lines; SHA 1e95ff22231e) - Capture current FRED closes to stdout; no historical/production admission. Main symbols: main.

cross_asset_client.py (185 lines; SHA fa42b5b6d9f5) - Bounded FRED daily-close capture for research Seat 28. Main symbols: CrossAssetClientError, NoRedirect.

ecb_calendar_backfill.py (227 lines; SHA b6ef1fd4b3b3) - Historical ECB policy calendar backfill and persistence coordination. Main symbols: ECBPolicyCalendarBackfillResult, ECBPolicyCalendarBackfill.

ecb_calendar_builder.py (310 lines; SHA c66fb05bca14) - Derives structured ECB meeting calendar records from official release evidence. Main symbols: ECBPolicyCalendarBuilder.

ecb_calendar_protocol.py (469 lines; SHA dbe3eecfd2c3) - Typed ECB meeting schedule/specification and validation logic. Main symbols: ECBPolicyMeetingSpec.

ecb_rate_backfill.py (359 lines; SHA 4b9839538e23) - Acquires historical ECB rate releases and stores bounded backfills. Main symbols: ECBRateBackfillResult, ECBRateBackfill.

ecb_rate_client.py (1058 lines; SHA 5cef3c8f3335) - ECB source client and strict validation/parser for policy rate history. Main symbols: ECBRateClient.

ecb_rate_protocol.py (305 lines; SHA 1fc92ca1f36b) - Defines release, observation and availability rules for ECB rate series. Main symbols: ECBRateSeriesSpec.

fed_calendar_artifact_intake.py (418 lines; SHA f24ef74be164) - F4.5.10: Offline, fail-closed intake of supplied FOMC evidence artifacts. Main symbols: FedCalendarArtifactIntakeError, FedCalendarArtifactNotReady.

fed_calendar_authenticity_requirements.py (378 lines; SHA 7845d0d51a87) - F4.5.11: Offline Fed calendar artifact authenticity verification requirements. Main symbols: FedCalendarAuthenticityRequirementsError, FedCalendarAuthenticityNotReady.

fed_calendar_client.py (355 lines; SHA 27bc4b93eb30) - F4.5.5a: Strict HTTPS acquisition of one pinned FOMC schedule document. Main symbols: FedCalendarAcquisitionError, _RejectRedirects.

fed_calendar_crypto_adapter.py (391 lines; SHA 38c1d5d9c765) - F4.5.14: non-executing RFC 3161 crypto-adapter handoff preflight. Main symbols: FedCryptoAdapterError, FedCryptoAdapterUnavailable.

fed_calendar_crypto_prerequisites.py (506 lines; SHA b7f3997ecdc7) - F4.5.12: Read-only prerequisites for trustworthy FOMC timestamp verification. Main symbols: FedCryptoPrerequisitesError, FedCryptoPrerequisitesNotReady.

fed_calendar_evidence_audit.py (210 lines; SHA 0aace0455d39) - F4.5.7: Pure, fail-closed FOMC schedule source-version evidence audit. Main symbols: FedCalendarEvidenceAuditError, FedCalendarEvidenceNotReady.

fed_calendar_extractor.py (506 lines; SHA aefa9e8981c3) - F4.5.6: Pure, fail-closed extraction of one Fed schedule acquisition. Main symbols: FedCalendarExtractionError, _VisibleLines.

fed_calendar_independent_evidence.py (420 lines; SHA 2a4144726307) - F4.5.9: Fail-closed catalog of claimed independent FOMC calendar evidence. Main symbols: FedCalendarIndependentEvidenceError, FedCalendarIndependentEvidenceNotReady.

fed_calendar_protocol.py (267 lines; SHA 79ece37412b0) - F4.5.4a: Offline, diagnostic FOMC regular-meeting schedule protocol. Main symbols: FedCalendarProtocolError, FedScheduledMeetingSpec.

fed_calendar_provenance_gaps.py (254 lines; SHA ee600b998adf) - F4.5.8: Read-only FOMC historical provenance gap assessment. Main symbols: FedCalendarProvenanceGapError, FedCalendarProvenanceNotReady.

fed_calendar_rfc3161_engine_prerequisites.py (532 lines; SHA 92a7bada7ef7) - F4.5.15: sealed, OFFLINE RFC 3161 engine provisioning/prerequisite inventory. Main symbols: FedRfc3161EnginePrerequisiteError, FedRfc3161EngineUnavailable.

fed_calendar_rfc3161_execution_isolation.py (539 lines; SHA 2b4602a2220a) - F4.5.16: fail-closed, offline RFC 3161 execution-isolation DESIGN boundary. Main symbols: FedRfc3161IsolationError, FedRfc3161IsolationUnavailable.

fed_calendar_rfc3161_local_verifier.py (510 lines; SHA c2b1de62fc74) - F4.5.18: OPT-IN executable RFC 3161 verification, quarantined from admission. Main symbols: LocalVerifierError, LocalVerifierUnavailable.

fed_calendar_rfc3161_runtime_attestation.py (599 lines; SHA b0ec1496f7d4) - F4.5.17: offline, fail-closed RFC 3161 runtime-attestation prerequisites. Main symbols: FedRfc3161RuntimeAttestationError, FedRfc3161RuntimeAttestationUnavailable.

fed_calendar_verifier_boundary.py (515 lines; SHA 34b7bc387457) - F4.5.13: Sealed RFC 3161 verification *execution boundary* for Fed calendar. Main symbols: FedVerifierBoundaryError, FedTrustedVerificationUnavailable.

fed_certification.py (289 lines; SHA 79b449a0f7a2) - F3.7: Federal Reserve evidence certification model. Main symbols: FedCertificationError, FedEvidenceResult.

fed_certified_backfill.py (244 lines; SHA 2653f173b300) - F3.7.4 Certified Federal Reserve backfill orchestration. Main symbols: FedCertifiedBackfillError, FedCertifiedBackfillResult.

fed_certified_storage.py (241 lines; SHA 9ca706c1690c) - F3.7.3a Federal Reserve certified persistence wrapper. Main symbols: FedCertifiedStorageError, FedCertifiedWriteResult.

fed_rate_alfred_client.py (669 lines; SHA 86e373cd22fd) - F3.4: read-only, fail-closed FRED/ALFRED v1 vintage acquisition. Main symbols: FedAlfredClientError, _NoRedirect.

fed_rate_backfill.py (347 lines; SHA adb84ed1278c) - F3.2: read-only Federal Reserve backfill planning and provenance preflight. Main symbols: FedBackfillPreflightError, FedSourceFingerprint.

fed_rate_candidate_repository.py (161 lines; SHA 6d89bcaf6eb3) - F5.1: read-only, causal Federal Reserve rate *candidate* repository. Main symbols: FedCandidateRepositoryError, FedRateCandidateRepository.

fed_rate_client.py (187 lines; SHA 71ae045ccd6e) - F2.1: strict official FOMC HTML acquisition, without policy extraction. Main symbols: FedSourceError, _RejectRedirect.

fed_rate_extractor.py (466 lines; SHA e6635ff7a24c) - F2.4: fail-closed extraction of original FOMC policy-rate evidence. Main symbols: FedEvidenceExtractionError, _VisibleHTML.

fed_rate_protocol.py (626 lines; SHA fc1026edef60) - F1: fail-closed, original-release FOMC target-range protocol. Main symbols: FedRateProtocolError, FedReleaseStamp.

fed_rate_vintage_protocol.py (340 lines; SHA b15ac64fba81) - F3.3.1: ALFRED date-granular policy-rate vintage corroboration. Main symbols: FedVintageProtocolError, FedAlfredVintageObservation.

fed_replay_lock.py (244 lines; SHA a42425912d6f) - F4.3.1: Pure, conservative Fed target-range replay selection. Main symbols: FedReplayLockError.

macro_repository.py (759 lines; SHA f0fb0a62e76b) - As-of queries for central bank rates and economic event calendar snapshots. Main symbols: CentralBankRateRepository, EconomicCalendarRepository.

macro_schemas.py (221 lines; SHA 85346a80d283) - Typed macroeconomic rate and calendar snapshot records with timestamps. Main symbols: CentralBankRateSnapshot, EconomicCalendarEventSnapshot.

macro_storage.py (386 lines; SHA 9cf706fffb0c) - Persists validated central bank and calendar records, with Fed-specific guardrails. Main symbols: MacroStorageResult, FedRatePersistenceBlocked.

Test modules

test_cross_asset_client.py (139 lines; SHA b9fed2f5a5e5) - Tests cross asset client code/contracts; 12 pytest-style test functions/methods, including aligned capture uses local availability not historical date; api key not in capture; credential echo is rejected Archived source only; current run status is not established by file presence.

test_ecb_calendar_backfill.py (665 lines; SHA e43f129e771a) - Procedural regression/fixture for ecb calendar backfill; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_ecb_calendar_builder.py (679 lines; SHA 9459cfcd05a4) - Procedural regression/fixture for ecb calendar builder; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_ecb_calendar_protocol.py (499 lines; SHA a111aa4ab622) - Procedural regression/fixture for ecb calendar protocol; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_ecb_rate_backfill.py (919 lines; SHA 233a92ec2e5a) - Procedural regression/fixture for ecb rate backfill; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_ecb_rate_client.py (933 lines; SHA 2f398758d3ec) - Procedural regression/fixture for ecb rate client; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_ecb_rate_protocol.py (383 lines; SHA ebf2eaa2617b) - Procedural regression/fixture for ecb rate protocol; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_fed_admission_integration.py (267 lines; SHA e2b3f3de693d) - Tests fed admission integration code/contracts; 4 pytest-style test functions/methods, including 01 corrobated evidence never reaches storage; 02 persistence approved evidence reaches storage; 03 tampered certification cannot exist Archived source only; current run status is not established by file presence.

test_fed_calendar_artifact_intake.py (432 lines; SHA 2ea7c159cb4c) - Tests fed calendar artifact intake code/contracts; 50 pytest-style test functions/methods, including 01 no artifacts is still a blocked report; 02 memento bytes match claim only locally; 03 opaque timestamp token not verified Archived source only; current run status is not established by file presence.

test_fed_calendar_authenticity_requirements.py (523 lines; SHA cbacee669d12) - Tests fed calendar authenticity requirements code/contracts; 52 pytest-style test functions/methods, including 01 memento bytes locally reconciled; 02 memento origin and custody require independent proof; 03 reported memento link is not authenticated Archived source only; current run status is not established by file presence.

test_fed_calendar_client.py (562 lines; SHA aaaef6bb7133) - Tests fed calendar client code/contracts; 25 pytest-style test functions/methods, including 01 capture raw bytes hash and observation; 02 request pins exact official url and headers; 03 transport is explicitly unverified when injected Archived source only; current run status is not established by file presence.

test_fed_calendar_crypto_adapter.py (470 lines; SHA 8ebb2d4f1df8) - Tests fed calendar crypto adapter code/contracts; 62 pytest-style test functions/methods, including 01 result is sealed report; 02 canonical boundary hash is bound; 03 exactly one timestamp target Archived source only; current run status is not established by file presence.

test_fed_calendar_crypto_prerequisites.py (483 lines; SHA acb1bc639375) - Tests fed calendar crypto prerequisites code/contracts; 54 pytest-style test functions/methods, including 01 one token target and upstream binding; 02 all council and historical authorizations false; 03 exact eleven unique check codes Archived source only; current run status is not established by file presence.

test_fed_calendar_evidence_audit.py (509 lines; SHA 38bd604e654f) - Tests fed calendar evidence audit code/contracts; 35 pytest-style test functions/methods, including 01 valid diagnostic reconciles eight meetings; 02 captured bytes digest is retained; 03 no independent history or admission Archived source only; current run status is not established by file presence.

test_fed_calendar_extractor.py (765 lines; SHA f8a75158a299) - Tests fed calendar extractor code/contracts; 37 pytest-style test functions/methods, including 01 exact eight candidate dates; 02 dst and statement time from existing protocol; 03 2027 rollover is explicit not ninth 2026 event Archived source only; current run status is not established by file presence.

test_fed_calendar_independent_evidence.py (431 lines; SHA 5268011fa434) - Tests fed calendar independent evidence code/contracts; 39 pytest-style test functions/methods, including 01 no external references remains blocked; 02 structurally valid claim is not proof; 03 claimed same bytes not authenticated Archived source only; current run status is not established by file presence.

test_fed_calendar_protocol.py (352 lines; SHA a1f197453311) - Tests fed calendar protocol code/contracts; 25 pytest-style test functions/methods, including 01 exact 2026 schedule from official release; 02 official label august 2024 1330 edt; 03 january winter statement nominal 1900 utc Archived source only; current run status is not established by file presence.

test_fed_calendar_provenance_gaps.py (544 lines; SHA 2883016fd883) - Tests fed calendar provenance gaps code/contracts; 32 pytest-style test functions/methods, including 01 receipt reports eight reconciled meetings; 02 body digest bound to prior audit; 03 original release label not historical proof Archived source only; current run status is not established by file presence.

test_fed_calendar_rfc3161_engine_prerequisites.py (669 lines; SHA f2ee3599024a) - Tests fed calendar rfc3161 engine prerequisites code/contracts; 89 pytest-style test functions/methods, including 01 frozen report type; 02 upstream manifest binding; 03 one target one reference Archived source only; current run status is not established by file presence.

test_fed_calendar_rfc3161_execution_isolation.py (670 lines; SHA b96e39bedbd3) - Tests fed calendar rfc3161 execution isolation code/contracts; 89 pytest-style test functions/methods, including 01 output is sealed report; 02 binding to canonical f4515 digest; 03 binding to original f4514 receipt Archived source only; current run status is not established by file presence.

test_fed_calendar_rfc3161_local_verifier.py (564 lines; SHA cb1854b6e46e) - Tests fed calendar rfc3161 local verifier code/contracts; 64 pytest-style test functions/methods, including 01 boundaries mirror f4516; 02 binary must be absolute path; 03 missing executable cannot be approved Archived source only; current run status is not established by file presence.

test_fed_calendar_rfc3161_runtime_attestation.py (746 lines; SHA 57e541eb80bb) - Tests fed calendar rfc3161 runtime attestation code/contracts; 102 pytest-style test functions/methods, including 01 frozen report model; 02 isolation receipt bound; 03 adapter receipt bound Archived source only; current run status is not established by file presence.

test_fed_calendar_verifier_boundary.py (549 lines; SHA 9824f880e040) - Tests fed calendar verifier boundary code/contracts; 68 pytest-style test functions/methods, including 01 canonical upstream binding; 02 one token one policy work order; 03 unique check codes Archived source only; current run status is not established by file presence.

test_fed_certification.py (244 lines; SHA d74bd893f608) - Tests fed certification code/contracts; 10 pytest-style test functions/methods, including 01 corroborated is valid; 02 causally available is valid; 03 intraday certified can support persistence Archived source only; current run status is not established by file presence.

test_fed_certified_backfill.py (181 lines; SHA 696223b0c153) - Tests fed certified backfill code/contracts; 4 pytest-style test functions/methods, including 01 corrobation creates non persistable certification; 02 persistence requires authorized certification; 03 storage wrapper is never bypassed Archived source only; current run status is not established by file presence.

test_fed_certified_storage.py (370 lines; SHA 9eb619a9f981) - Tests fed certified storage code/contracts; 15 pytest-style test functions/methods, including 01 approved certification reaches storage; 02 corroborated is blocked; 03 causally available is blocked Archived source only; current run status is not established by file presence.

test_fed_f3_integrity_boundary.py (98 lines; SHA e2dd0b8f329a) - Tests fed f3 integrity boundary code/contracts; 6 pytest-style test functions/methods, including 01 real synthetic source pair never mints integrity pass; 02 corroborated source does not forward to injected storage; 03 fabricated approved object cannot enter actual db writer Archived source only; current run status is not established by file presence.

test_fed_rate_alfred_client.py (776 lines; SHA 1444f442e4d8) - Tests fed rate alfred client code/contracts; 27 pytest-style test functions/methods, including 01 exact endpoint query and header contract; 02 complete pair and raw sha256 fingerprints; 03 aware fetch clocks normalized to utc Archived source only; current run status is not established by file presence.

test_fed_rate_backfill.py (437 lines; SHA 996745e9e048) - Tests fed rate backfill code/contracts; 20 pytest-style test functions/methods, including 01 complete pair and decision identity; 02 historical persistence explicitly prohibited; 03 three exact verified source fingerprints Archived source only; current run status is not established by file presence.

test_fed_rate_client.py (287 lines; SHA 350184a90323) - Tests fed rate client code/contracts; 14 pytest-style test functions/methods, including 01 official url construction; 02 no network at construction; 03 verified tls and request headers Archived source only; current run status is not established by file presence.

test_fed_rate_extractor.py (566 lines; SHA 5c5ebf251e37) - Tests fed rate extractor code/contracts; 17 pytest-style test functions/methods, including 01 september three document evidence; 02 normalized snapshots and causal boundary; 03 ignore dissent and non target operational rates Archived source only; current run status is not established by file presence.

test_fed_rate_protocol.py (599 lines; SHA f8b8d3b6a35d) - Procedural regression/fixture for fed rate protocol; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_fed_rate_storage_gate.py (188 lines; SHA 8a9ff962c93e) - Tests fed rate storage gate code/contracts; 11 pytest-style test functions/methods, including 01 fed lower blocked before session creation; 02 fed upper

blocked before session creation; 03 mixed batch blocks before any database action Archived source only; current run status is not established by file presence.

test_fed_rate_vintage_protocol.py (571 lines; SHA a3d7ded9288b) - Tests fed rate vintage protocol code/contracts; 23 pytest-style test functions/methods, including 01 canonical match has expected identity and dates; 02 matching vintage never becomes historical certification; 03 official series contract and strict order Archived source only; current run status is not established by file presence.

test_fed_replay_lock.py (532 lines; SHA 88306bc5553f) - Tests fed replay lock code/contracts; 26 pytest-style test functions/methods, including 01 empty is allowed; 02 complete pair at ingestion boundary; 03 release before effective date is allowed Archived source only; current run status is not established by file presence.

test_macro_context.py (861 lines; SHA ab92d7418cd9) - Procedural regression/fixture for macro context; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_macro_repository.py (1186 lines; SHA 901bec0c01af) - Procedural regression/fixture for macro repository; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_macro_schemas.py (519 lines; SHA a7ff9f0893a8) - Procedural regression/fixture for macro schemas; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_macro_storage.py (1442 lines; SHA 4c3631906837) - Procedural regression/fixture for macro storage; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_macro_storage_models.py (389 lines; SHA 916dfcaed7fe) - Procedural regression/fixture for macro storage models; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

12.10. backend/main.py/ - 1 Python files (1 non-test; 0 tests)

Application and supporting modules

main.py (24 lines; SHA e646045f36ba) - Prints project identity/version/environment banner; it is not an orchestration scheduler or trading engine. Main symbols: main.

Test modules

12.11. backend/market_state/ - 19 Python files (11 non-test; 8 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

candle_repository.py (314 lines; SHA bff58a7a08db) - Fetches validated candle sequences for market-state calculations. Main symbols: CandleRepository.

liquidity_engine.py (404 lines; SHA be44d43d3154) - Calculates spread/liquidity and price-level evidence from market data. Main symbols: LiquidityEngineResult, LiquidityEngine.

market_state_engine.py (525 lines; SHA a88e68017887) - Aggregates trend, volatility, momentum, structure, liquidity and session engine outputs. Main symbols: UnifiedMarketStateResult, MarketStateEngine.

momentum_engine.py (686 lines; SHA e96d4528aa72) - Calculates directional momentum, indicators and acceleration metrics from candles. Main symbols: MomentumEngineResult, MomentumEngine.

schemas.py (295 lines; SHA 8024c7434601) - Typed structured market-state records, enums and validation contracts. Main symbols: MarketDirection, VolatilityRegime.

session_engine.py (378 lines; SHA 2cb59f6ed1c4) - Determines London/New York/session state with time windows and session metrics. Main symbols: SessionEngineResult, SessionEngine.

structure_engine.py (749 lines; SHA 3027fa851e15) - Identifies swing points, breaks and price structure transitions in candle history. Main symbols: SwingPoint, StructureEngineResult.

tick_repository.py (114 lines; SHA 10330942bb1f) - Retrieves historical tick observations as input to market-state engines. Main symbols: TickRepository.

trend_engine.py (701 lines; SHA bac354f7c1d3) - Computes trend direction and strength from candle-price history. Main symbols: TrendEngineResult, TrendEngine.

volatility_engine.py (601 lines; SHA 3315e4952d6f) - Computes ATR, volatility measures, spread/volatility conditions and regime labels. Main symbols: VolatilityEngineResult, VolatilityEngine.

Test modules

test_liquidity_engine.py (565 lines; SHA bc93d2a9f8a6) - Procedural regression/fixture for liquidity engine; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_market_state_engine.py (444 lines; SHA f82338eacbc0) - Procedural regression/fixture for market state engine; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_momentum_engine.py (394 lines; SHA ef4068622af1) - Procedural regression/fixture for momentum engine; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_repository.py (202 lines; SHA 555584f32eee) - Procedural regression/fixture for repository; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_session_engine.py (476 lines; SHA 3f719590269e) - Procedural regression/fixture for session engine; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_structure_engine.py (489 lines; SHA 23511b0740b6) - Procedural regression/fixture for structure engine; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_trend_engine.py (349 lines; SHA 3de68587a17c) - Procedural regression/fixture for trend engine; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_volatility_engine.py (426 lines; SHA 5e1b88e752e8) - Procedural regression/fixture for volatility engine; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

12.12. backend/models/ - 55 Python files (40 non-test; 15 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

artifact_contracts.py (143 lines; SHA dd3a5ac4000f) - Defines model family and versioned model metadata/artifact integrity contracts. Main symbols: ModelFamily, ModelArtifactMetadata.

audit_historical_backfill.py (483 lines; SHA 573eaf327f39) - Read/report-oriented historical MT5-versus-database count and window audit tool. Main symbols: parse_date, parse_arguments.

audit_ml_dataset.py (207 lines; SHA ef25977d5cb1) - Audits stored training samples, dates, distribution and dataset integrity. Main symbols: parse_date, parse_arguments.

calibrate_neural_ensemble.py (919 lines; SHA 78fc21f6233a) - Fits/evaluates temperature calibration of frozen neural ensemble probabilities. Main symbols: write_json, metrics_for_probabilities.

cleanup_historical_range.py (247 lines; SHA 024e650eeac9) - Historical DB cleanup utility; potentially destructive and never part of read-only review. Main symbols: parse_date, parse_arguments.

compare_hmm_grid.py (1225 lines; SHA b0689dae9cb3) - Compares HMM candidate state counts/seeds and selects a documented experimental candidate. Main symbols: Candidate.

compare_neural_xgboost_validation.py (768 lines; SHA dad7bf73ebfc) - Measures validation metrics and calibration for neural ensemble versus XGBoost. Main symbols: multiclass_brier_score, expected_calibration_error.

compatibility.py (116 lines; SHA 139a2c67480b) - Checks feature schema/model artifact compatibility before inference. Main symbols: CompatibilityResult, ModelCompatibilityChecker.

dataset_models.py (114 lines; SHA 9e6da0986165) - Defines SQL/ORM typed training sample records. Main symbols: MLTrainingSample.

dataset_partitioning.py (139 lines; SHA 4a12d9f9a1df) - Explicit chronological training, validation and held-out split boundaries. Main symbols: ChronologicalSplitPlan, FeatureBatch.

evaluate_baseline_models.py (225 lines; SHA 57d36a490d7d) - Evaluates baseline classifier performance on designated frozen data partitions. Main symbols: encode_labels, evaluate_split.

evaluate_hmm_test.py (1543 lines; SHA 5c15832e7a66) - Reads selected HMM candidate and evaluates frozen reserved test split. Main symbols: read_json, write_json.

evaluate_neural_ensemble_test.py (973 lines; SHA 417f232eb7d4) - Final reserved test evaluation of selected frozen neural ensemble. Main symbols: read_json, write_json.

evaluate_neural_ensemble_validation.py (1417 lines; SHA 050a3fd0ee1d) - Evaluates a calibrated neural ensemble on validation split, not deployment returns. Main symbols: read_json, write_json.

feature_builder.py (580 lines; SHA 387d7f453d23) - Builds versioned EURUSD M5 ML features with windows and availability constraints. Main symbols: MLFeatureBuilder.

feature_contracts.py (89 lines; SHA a197313670e9) - Defines typed, versioned ML feature vector contract. Main symbols: ModelFeatureVector.

historical_backfill.py (911 lines; SHA 63db8bfa9d3c) - Bounded MT5 historical data backfill into stored candle/tick data. Main symbols: parse_date, parse_arguments.

historical_dataset_builder.py (531 lines; SHA 2e0870cf8252) - Creates historic ML training dataset from validated candles and feature windows. Main symbols: HistoricalDatasetResult, HistoricalDatasetBuilder.

historical_replay.py (1253 lines; SHA 67c3bf8abd42) - Replays historical market/macro observations under as-of availability constraints. Main symbols: HistoricalReplayResult, HistoricalReplayBuilder.

hmm_core.py (774 lines; SHA 6ae2b64b2e48) - Pure HMM state probabilities, Gaussian emission and causal filtering math. Main symbols: HMMFeatureStandardizer, DiagonalGaussianHMMParameters.

hmm_model_registry.py (750 lines; SHA 3cd26301ccf5) - Loads and validates frozen HMM parameters, metadata and checksums for inference. Main symbols: HMMModelRegistry.

hmm_observation_builder.py (956 lines; SHA 96d9d3ebd62c) - Builds ordered, standardized observations and alignment for contextual HMM inference. Main symbols: HMMObservationSequence, HMMObservationBuilder.

hmm_training_dataset.py (1109 lines; SHA 249c6ba79598) - Creates versioned HMM training/validation/test sequences under causal time windows. Main symbols: HMMAlignedSplit, HMMDevelopmentDataset.

labeling.py (129 lines; SHA 2286b15f10e2) - Labels future horizon price movements into classification targets. Main symbols: DirectionLabeler.

model_registry.py (27 lines; SHA 5149916e23b0) - Loads/identifies frozen baseline/tree model artifacts and metadata. Main symbols: ModelRegistry.

neural_model_registry.py (1020 lines; SHA 9364c7272698) - Validates neural ensemble members, normalization, calibrator and metadata. Main symbols: NeuralEnsembleInference, NeuralEnsembleArtifact.

neural_network_architecture.py (503 lines; SHA 8b98b0cafce8) - PyTorch architecture definition for nonlinear forecast models and feature gates. Main symbols: NeuralNetworkConfig, FeatureGate.

neural_training_engine.py (888 lines; SHA aa6e4206c6ad) - Training/evaluation engine and deterministic metric collection for neural experiments. Main symbols: NeuralTrainingConfig, EvaluationMetrics.

neural_training_utils.py (1055 lines; SHA 4680016d2505) - Normalization, ordered splits, batching and data-preparation helpers for neural training. Main symbols: FeatureStandardizer, ChronologicalSplit.

populate_ml_dataset.py (336 lines; SHA 7ea9913297c8) - Creates/stores historical feature-label training rows; a DB write utility. Main symbols: PopulationStats.

probe_mt5_history.py (632 lines; SHA 4f61c8439c0a) - MT5 history-probing CLI for historical range coverage; can connect to broker. Main symbols: parse_arguments, broker_now_utc.

promote_hmm_v1.py (743 lines; SHA bb0d02193d48) - Frozen HMM promotion into registry with integrity/metadata checks. Main symbols: read_json, write_json.

promote_neural_network_v1.py (648 lines; SHA 03107d5d1d4f) - Validated neural ensemble promotion to model registry, potentially writes artifacts. Main symbols: read_json, write_json.

run_neural_seed_experiment.py (77 lines; SHA 654d0481eb54) - CLI for neural seed-specific experiments; triggers model fitting. Main symbols: main.

train_baseline_models.py (396 lines; SHA 6847f2f7ed91) - Fits logistic/XGBoost baseline classifiers and writes model/metric artifacts. Main symbols: encode_labels, evaluate.

train_hmm_experiment.py (1546 lines; SHA 696b082cc616) - Fits and evaluates contextual HMM candidates; writes experiment metadata. Main symbols: json_safe, write_json.

train_neural_network_experiment.py (1429 lines; SHA debd11d0bb76) - Fits and evaluates neural classifiers; writes experimental checkpoints. Main symbols: class_distribution, collect_probabilities.

training_contracts.py (194 lines; SHA 043e74401fd3) - Typed label, horizon, provenance and training-sample contracts. Main symbols: TargetLabel, LabelSpecification.

training_dataset.py (121 lines; SHA 52f466f750ef) - Loads versioned training datasets from stored sample records. Main symbols: TrainingDataset, TrainingDatasetLoader.

Test modules

test_dataset_partitioning.py (83 lines; SHA 1845333d9678) - Tests dataset partitioning code/contracts; 2 pytest-style test functions/methods, including split boundaries and purge; bounded feature batch geometry Archived source only; current run status is not established by file presence.

test_feature_builder.py (248 lines; SHA 308be924380a) - Procedural regression/fixture for feature builder; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_fed_replay_builder_integration.py (320 lines; SHA f7a849c519c6) - Tests fed replay builder integration code/contracts; 11 pytest-style test functions/methods, including 01 legacy build default has empty fed context; 02 explicit empty fed input accepted; 03 legacy seven positional arguments unchanged Archived source only; current run status is not established by file presence.

test_historical_dataset_builder.py (612 lines; SHA 5822c7df95e0) - Procedural regression/fixture for historical dataset builder; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_hmm_core.py (638 lines; SHA 262b8a12a073) - Procedural regression/fixture for hmm core; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_hmm_model_registry.py (449 lines; SHA da02e155c7bc) - Procedural regression/fixture for hmm model registry; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_hmm_observation_builder.py (1001 lines; SHA 0587a5989db3) - Procedural regression/fixture for hmm observation builder; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_hmm_training_dataset.py (513 lines; SHA 20f23cdb5f98) - Procedural regression/fixture for hmm training dataset; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_model_contracts.py (438 lines; SHA 24c3071a6e2d) - Procedural regression/fixture for model contracts; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_model_registry.py (37 lines; SHA 23f819ca79bc) - Procedural regression/fixture for model registry; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_neural_model_registry.py (460 lines; SHA 68ed2e78a8c2) - Procedural regression/fixture for neural model registry; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_neural_network_architecture.py (251 lines; SHA 99f18301c993) - Procedural regression/fixture for neural network architecture; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_neural_training_engine.py (403 lines; SHA 131cc1d50271) - Procedural regression/fixture for neural training engine; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_neural_training_utils.py (443 lines; SHA ba11060d2424) - Procedural regression/fixture for neural training utils; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

test_training_dataset.py (47 lines; SHA a3c623414bc2) - Procedural regression/fixture for training dataset; no test_* function detected (inspect its main() and assertions). Archived source only; current run status is not established by file presence.

12.13. backend/runtime/ - 10 Python files (7 non-test; 3 tests)

Application and supporting modules

__init__.py (0 lines; SHA e3b0c44298fc) - Package initializer; empty in archived snapshot.

capture_forward_feed.py (110 lines; SHA 347c549527c1) - Bounded forward UTC capture into separate tables; no Council or broker orders. Main symbols: ForwardFeedRecorder.

check_market_data.py (98 lines; SHA aef4e8dd73b0) - Read-only MT5/DB health probe. Never ingests data or sends orders. Main symbols: inspect_terminal, inspect_database.

create_clock_policy.py (78 lines; SHA 5a5c4c54ebd1) - Create a server-bound forward lease from recent probe evidence. No DB writes. Main symbols: from_evidence, main.

market_data_supervisor.py (684 lines; SHA 69bb6556f296) - Supervises independent market-data workers, restart policy, health and graceful shutdown; operational, not a read-only command. Main symbols: WorkerSpec, WorkerState.

probe_mt5_time_domain.py (122 lines; SHA 4c322bd4fbae) - Bounded, read-only evidence for MT5 payload and query timestamp conventions. Main symbols: _instant, probe.

process_lock.py (245 lines; SHA 9ad023628f2c) - Single-instance process lock to prevent duplicate data workers. Main symbols: SingleInstanceLock.

Test modules

test_create_clock_policy.py (58 lines; SHA 66cf3bdbb96f) - Tests create clock policy code/contracts; 4 pytest-style test functions/methods, including recent complete evidence creates a forward only lease; stale incomplete or unknown identity cannot create policy; query domain must be independently demonstrated Archived source only; current run status is not established by file presence.

test_forward_supervisor.py (46 lines; SHA 9f0cd7289afa) - Tests forward supervisor code/contracts; 5 pytest-style test functions/methods, including no policy keeps the existing two workers; policy mode has one forward worker and no legacy pair; worker arguments are passed without a shell Archived source only; current run status is not established by file presence.

test_market_data_health.py (78 lines; SHA 136abdd6d644) - Tests market data health code/contracts; 7 pytest-style test functions/methods, including connected current quote is healthy; shifted raw quote remains quarantined; ipc failure does not expose error message Archived source only; current run status is not established by file presence.

13. Post-archive Stage 2-5 implementation additions - separate provenance

The October 27 ZIP predates the user-confirmed Stage 4-5 commits. Do **not** infer that their absence from the archive means the files are missing in the current Windows checkout. The later stage entries below are based on the original project development conversation, code supplied during the work, and user-observed Windows test/Git evidence. The final two Stage 5 modules were also available directly in the supplied workspaces. An authoritative current-branch inventory should eventually reconcile all these paths byte-for-byte.

13.1 Stage 2 and 3 engineering tests visible in the latest Windows regression

- **backend/data_engine/test_phase2_acceptance.py — Implementation/test purpose from development history:** Phase 2 forward research engineering acceptance checks
- **backend/data_engine/test_forward_research_evaluation.py — Implementation/test purpose from development history:** Offline forward research evaluation readiness

- **backend/data_engine/test_forward_cycle_readiness.py — Implementation/test purpose from development history:** Clock/forward cycle readiness and blockers
- **backend/data_engine/test_forward_source_lineage.py — Implementation/test purpose from development history:** Forward source ancestry and availability
- **backend/data_engine/test_forward_evidence_health.py — Implementation/test purpose from development history:** Forward evidence integrity and health
- **backend/runtime/test_phase3_acceptance.py — Implementation/test purpose from development history:** Phase 3 consolidated research acceptance
- **backend/runtime/test_forward_prospective_evaluation.py — Implementation/test purpose from development history:** Prospective prediction/outcome evaluation constraints
- **backend/runtime/test_forward_market_input_lineage.py — Implementation/test purpose from development history:** Market input source lineages and causal provenance
- **backend/runtime/test_forward_source_provenance.py — Implementation/test purpose from development history:** External/forward source provenance checks
- **backend/runtime/test_forward_recovery_evidence.py — Implementation/test purpose from development history:** Restart/recovery evidence requirements
- **backend/runtime/test_forward_collector_preflight.py — Implementation/test purpose from development history:** Forward collector preflight and HOLD behavior

These test paths appeared in completed Windows regression commands. The current associated production module contents were not all available in the October 27 source ZIP and have **not** been independently byte-inspected here. Do not invent their public entrypoints or commit ancestry.

13.2 Stage 4 read-only inspector/test pairs

- **backend/runtime/stage4_collector_forensics.py — Scope:** Collector preflight, historical forensics and provenance gaps **Commit:** 00f2618
- **backend/runtime/stage4_broker_clock_evidence.py — Scope:** Original clock-policy/broker-source evidence and no-rebind rules **Commit:** 9550bc7
- **backend/runtime/stage4_forward_receipt_recovery.py — Scope:** Forward receipt structure, capture chronology and recovery evidence **Commit:** 7ef1d0e
- **backend/runtime/stage4_council_input_provenance.py — Scope:** Council input source availability and derived lineage **Commit:** 375a2fd
- **backend/runtime/stage4_prospective_evidence_custody.py — Scope:** Prediction, outcome, cost and holdout custody requirements **Commit:** 1e27212
- **backend/runtime/stage4_final_handover.py — Scope:** Consolidated Stage 4 local source/custody handover **Commit:** 761f60f

Each pairs with, e.g., stage4_collector_forensics.py -> test_stage4_collector_forensics.py. These modules are read-only engineering inspectors, not live admission or source-issuing services. The Stage 4 source bytes are not included in the older archive; validate against the current checkout before modifying.

13.3 Stage 5 read-only inspector/test pairs

- **stage5_evidence_intake.py / test_stage5_evidence_intake.py — What they implement and what they explicitly do not prove:** Strict candidate evidence inventory, local trust levels and initial external-gate reconciliation; no proof from self-declared metadata **Commit:** 1031a5b

- **stage5_broker_clock_attestation.py / test_stage5_broker_clock_attestation.py — What they implement and what they explicitly do not prove:** Original broker identity/UTC/probe reconciliation; historic 3 observations, 7,200-second wall convention and pin mismatch; no independent broker certification **Commit:** 6612b09
- **stage5_capture_recovery_readiness.py / test_stage5_capture_recovery_readiness.py — What they implement and what they explicitly do not prove:** Reinspect 10 stored manifests, optionally hash genuine ordered original source lists, validate a proposed new-epoch plan without activating it; no new collector run **Commit:** 10b9de6
- **stage5_council_source_causality.py / test_stage5_council_source_causality.py — What they implement and what they explicitly do not prove:** Static 31-seat/13-family AST inspection, versioned release chronology, revisions and feature dependency DAG; no Council execution or real source certification **Commit:** 925eced
- **stage5_prospective_evidence_readiness.py / test_stage5_prospective_evidence_readiness.py — What they implement and what they explicitly do not prove:** Predecision candidate registration, post-horizon outcome, claimed costs and pre-frozen holdout checks; no independent evidence or performance claim **Commit:** dcccf0b
- **stage5_final_research_handover.py / test_stage5_final_research_handover.py — What they implement and what they explicitly do not prove:** Five-commit ancestry and source-blob checks, optional saved report hash checking, 11 external gate summary and deterministic HOLD report; **does not self-certify the last batch Commit: fb1c643**

All pairs above live under backend/runtime/. After each Stage 5 batch, user Windows tests passed and the corresponding new source/test pair was committed and pushed. The final Stage 5 suite result was **847 passed, 9 skipped, 61 subtests passed**. The last handover inspector itself deliberately printed PRIOR_20_GIT_CHECKPOINTS_VERIFIED_PENDING_FINAL_WINDOWS_TESTS before the user executed and pushed the last batch; subsequent logged test and push output establish 24/24 engineering completion. Do not misread its old pre-commit wording as an unresolved test failure.

14. Non-Python archive artifacts and support material

14.1 Dated engineering documents in the original ZIP

- **docs/PROJECT_AUDIT_2026-10-27.txt — Role and reading caution:** Earlier milestone audit while Seat 25 remained in progress; **superseded** by later Council consolidation
- **docs/BLUEPRINT_COMPARISON_2026-10-27.txt — Role and reading caution:** Gap mapping from implementation to original v1.1 blueprint at that date
- **docs/COUNCIL_IMPLEMENTATION_2026-10-27.txt — Role and reading caution:** Seats 25-31 research implementation before final canonical consolidation
- **docs/COUNCIL_CONSOLIDATION_2026-10-27.txt — Role and reading caution:** Authoritative Oct 27 31-seat registry/runner consolidation checkpoint
- **docs/ADAPTATION_MILESTONE_1_2026-10-27.txt — Role and reading caution:** Append-only learning memory and delayed-outcome implementation details
- **docs/CLOCK_EPOCH_MILESTONE_2026-10-27.txt — Role and reading caution:** Isolated UTC feed, original epoch/pinned policy and bounded capture proof
- **docs/clock_profiles/hfm_forward_20261027.json — Role and reading caution:** Original historical clock policy (preserve bytes and bound epoch)

- **docs/clock_profiles/hfm_probe_evidence_20261027.json — Role and reading caution:** Original local timestamp-probe evidence; independent UTC not established

The updated **Research-First Blueprint v1.2** exists as the separately supplied 12-page PDF plus an editable Markdown amendment. This atlas is supplemental documentation; neither document silently replaces source policy or independently clears an evidence gate.

14.2 Models, artifacts, logs and generated files

backend/models/registry/: frozen XGBoost (.pkl), logistic baseline, neural (member_11.pt, member_29.pt, member_47.pt), HMM (parameters.npz) and metadata/evaluation/checksum documents. **Never unpickle or load untrusted binary artifacts merely to document them.**

backend/models/experiments/: HMM k=3,4,5 seed investigations (including 11, 29, 47), neural-member and calibrated ensemble experiment metrics, state diagnostics, final decisions and selected test outputs. Historical evaluation is not a new independent holdout.

backend/models/dataset_manifest.json: historical dataset/source/split metadata. Treat as an archival reference, not a license to reuse test observations for tuning.

logs/: dated FOMC audits, dataset/clock/MT5 probes, Council/memory/forward feed test outputs and zipped milestone source snapshots. logs/synthetic_council_memory.sqlite3 is a **synthetic proof store**, not a production prediction ledger. logs/audit_pdf_dependencies/ is vendored audit tooling, not ORACLE core source.

venv/: bundled Windows environment, including pip/vendor packages and executables. Excluded from the first-party AST inventory. __pycache__/, .pytest_cache/, compiled .pyc and .git/ are generated/internal.

.env: private runtime configuration. **Do not reproduce, upload, check into Git or reveal contents.** .env.example contains only the expected environment variable names; requirements and locked versions are build metadata, not proof of repeatable deployment.

15. Exact transition point and next action

Where we are: Stage 5 **24/24 engineering completed and pushed** at fb1c643; Windows regression **847 passed, 9 skipped, 61 subtests**; original historical epoch preserved; **0/11 independent external gates** and **0/13 Council source families** attested. The system is **FINAL=HOLD**. No further stage has been started here.

Before Stage 6 implementation: make this atlas and its file inventory available to future chats and place the Markdown copy in docs/ if desired, then check the actual Windows branch for changes made since fb1c643. When Stage 6 starts, begin with offline **P6-01..P6-04 research governance/evidence contracts**. Use the same tested four-checkpoint, two-file batch workflow where it fits, with separate Windows verification and scoped Git commit. Update the atlas immediately after that future commit.

This atlas is a versioned documentation handover, not a software or broker authorization.

16. Complete auxiliary model and audit-file register (archive only)

This inventory supplements the 316-file Python catalog. It lists the **107 non-Python model/dataset files** and the **historical root logs/ files** from the October 27 archive. Binary artifacts were inventoried by filename, byte size and SHA prefix **without deserializing or executing them**. JSON evidence is treated as locally stored material, not externally attested truth. Vendored logs/audit_pdf_dependencies/ and bundled venv/ libraries are excluded as third-party packages.

16.1 Archived backend/models/ dataset, registry and experiments

backend/models/dataset_manifest.json (1137 bytes; SHA 21952923306b) - Historic dataset/split/features provenance manifest.

backend/models/experiments/hmm_eurusd_m5_v1_final_decision.json (1951 bytes; SHA 51c11e04aa58) - Frozen HMM candidate selection record.

backend/models/experiments/hmm_eurusd_m5_v1_grid_comparison.json (14260 bytes; SHA ae88aaec824a) - Comparison of HMM candidate state counts and seeds.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_11/metadata.json (866 bytes; SHA 4b780df9876b) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_11/state_diagnostics.json (2235 bytes; SHA ff68f617e443) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_11/state_outcome_probabilities.json (1172 bytes; SHA d1160f92e8b7) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_11/training_metrics.json (22283 bytes; SHA 970159010efb) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_11/transition_matrix.json (430 bytes; SHA b41ad33c43f3) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_11/validation_metrics.json (688 bytes; SHA 9b11336fc492) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_29/metadata.json (866 bytes; SHA 1cf0a9009799) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_29/state_diagnostics.json (2239 bytes; SHA 6111641fc984) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_29/state_outcome_probabilities.json (1175 bytes; SHA 28223bcee903) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_29/training_metrics.json (23636 bytes; SHA a4bd01e70d3b) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_29/transition_matrix.json (435 bytes; SHA 4e3845ba4b9f) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_29/validation_metrics.json (688 bytes; SHA 9cc5e7b8cb1d) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_47/metadata.json (866 bytes; SHA 3a892598afab) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_47/state_diagnostics.json (2240 bytes; SHA 723d31dafd29) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_47/state_outcome_probabilities.json (1171 bytes; SHA 86e744e336e7) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_47/training_metrics.json (24228 bytes; SHA e3bba459ed2c) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_47/transition_matrix.json (432 bytes; SHA b9a29f7e7c21) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k3_seed_47/validation_metrics.json (689 bytes; SHA 629d1e693934) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_11/metadata.json (866 bytes; SHA 804f0224dba8) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_11/state_diagnostics.json (2846 bytes; SHA 2af856e00950) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_11/state_outcome_probabilities.json (1509 bytes; SHA 8621ce13d64a) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_11/training_metrics.json (9841 bytes; SHA 867cc9bb3dd1) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_11/transition_matrix.json (672 bytes; SHA a168c59c806c) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_11/validation_metrics.json (691 bytes; SHA 98f995b4b1a0) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_29/metadata.json (866 bytes; SHA 98b2b995047e) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_29/state_diagnostics.json (2841 bytes; SHA ae8763728532) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_29/state_outcome_probabilities.json (1515 bytes; SHA de632a6064b9) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_29/training_metrics.json (10809 bytes; SHA 05160cb5fec9) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_29/transition_matrix.json (672 bytes; SHA 09fe01eca1c6) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_29/validation_metrics.json (695 bytes; SHA 224e26bacab6) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_47/metadata.json (866 bytes; SHA 7c375e9b7a9c) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_47/state_diagnostics.json (2842 bytes; SHA ac1c05246a3b) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_47/state_outcome_probabilities.json (1517 bytes; SHA 8adbc480cd71) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_47/training_metrics.json (11210 bytes; SHA af4d9f2280ef) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_47/transition_matrix.json (668 bytes; SHA 8030095c19b4) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k4_seed_47/validation_metrics.json (695 bytes; SHA 7280e5318988) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_11/candidate_parameters.npz (2643 bytes; SHA 4383de12c975) - Numerical arrays for HMM or normalization; not loaded in this review.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_11/final_test_evaluation.json (3157 bytes; SHA ad4f84bc8e9a) - Previously consumed held-out test evaluation; do not rerun during documentation.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_11/metadata.json (866 bytes; SHA 7988ecfdb061) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_11/state_diagnostics.json (3423 bytes; SHA 595378392d93) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_11/state_outcome_probabilities.json (1853 bytes; SHA beb6f5a44fde) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_11/training_metrics.json (52618 bytes; SHA a5be37b8230e) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_11/transition_matrix.json (966 bytes; SHA 32238791b61a) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_11/validation_metrics.json (689 bytes; SHA de7e392195c8) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_29/metadata.json (866 bytes; SHA 954382817cd8) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_29/state_diagnostics.json (3452 bytes; SHA 0f241694f702) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_29/state_outcome_probabilities.json (1857 bytes; SHA 9d398e9c3be1) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_29/training_metrics.json (26942 bytes; SHA 5e29323e3f18) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_29/transition_matrix.json (968 bytes; SHA 8f95d7a0736e) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_29/validation_metrics.json (690 bytes; SHA 1546164dbff9) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_47/metadata.json (866 bytes; SHA 23b65e1fd2bb) - Selected training/model configuration and artifact metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_47/state_diagnostics.json (3426 bytes; SHA 77e01f1cd0d1) - HMM latent-state diagnostic summaries.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_47/state_outcome_probabilities.json (1859 bytes; SHA 392304f1e089) - Observed class distributions conditioned on HMM state.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_47/training_metrics.json (52999 bytes; SHA 694e6fdb42a1) - Training partition performance metadata.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_47/transition_matrix.json (966 bytes; SHA 438e549c627a) - HMM state transition probabilities.

backend/models/experiments/hmm_eurusd_m5_v1_k5_seed_47/validation_metrics.json (688 bytes; SHA 8ef406258a90) - Validation partition performance and calibration data.

backend/models/experiments/hmm_eurusd_m5_v1_selection.json (1636 bytes; SHA 4d6652dd9778) - HMM registry/promotion selection record.

backend/models/experiments/neural_network_eurusd_m5_v1_ensemble_11_29_47/calibration.json (456 bytes; SHA d31d9bb82392) - Calibration coefficients/method selected for neural ensemble.

backend/models/experiments/neural_network_eurusd_m5_v1_ensemble_11_29_47/calibration_fit_diagnostics.json (654 bytes; SHA 7ec80dd6e309) - Diagnostics of neural ensemble calibration fit.

backend/models/experiments/neural_network_eurusd_m5_v1_ensemble_11_29_47/disagreement_metrics.json (437 bytes; SHA ed3f73ea7855) - Historical neural ensemble member disagreement measurements.

backend/models/experiments/neural_network_eurusd_m5_v1_ensemble_11_29_47/ensemble_manifest.json (806 bytes; SHA d0666d520e1e) - Defines selected neural ensemble members and frozen references.

backend/models/experiments/neural_network_eurusd_m5_v1_ensemble_11_29_47/final_test_evaluation.json (4181 bytes; SHA 351b19d26a9b) - Previously consumed held-out test evaluation; do not rerun during documentation.

backend/models/experiments/neural_network_eurusd_m5_v1_ensemble_11_29_47/latency.json (245 bytes; SHA 10f7e6b804c4) - Model inference latency measurements in experimental environment.

backend/models/experiments/neural_network_eurusd_m5_v1_ensemble_11_29_47/member_validation_metrics.json (4187 bytes; SHA f92e9548ed8a) - Per-neural-member validation measurements.

backend/models/experiments/neural_network_eurusd_m5_v1_ensemble_11_29_47/validation_metrics.json (1249 bytes; SHA 387538f05692) - Validation partition performance and calibration data.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_11/architecture.json (184 bytes; SHA 3fb3df326afe) - Neural model architecture and dimensions.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_11/latency.json (230 bytes; SHA 16f972d2b5b8) - Model inference latency measurements in experimental environment.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_11/metadata.json (1622 bytes; SHA fb65fa8e6de2) - Selected training/model configuration and artifact metadata.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_11/training_config.json (248 bytes; SHA cb52f843949c) - Seed-specific training hyperparameters and data configuration.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_11/training_history.json (7879 bytes; SHA 616153047084) - Recorded neural training iterations and loss/metric history.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_11/validation_metrics.json (2183 bytes; SHA ea0912f3247d) - Validation partition performance and calibration data.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_29/architecture.json (184 bytes; SHA 3fb3df326afe) - Neural model architecture and dimensions.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_29/latency.json (220 bytes; SHA 0318965acbbe) - Model inference latency measurements in experimental environment.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_29/metadata.json (1623 bytes; SHA d2750b2ed65a) - Selected training/model configuration and artifact metadata.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_29/training_config.json (248 bytes; SHA cb52f843949c) - Seed-specific training hyperparameters and data configuration.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_29/training_history.json (8376 bytes; SHA 5946ba04507f) - Recorded neural training iterations and loss/metric history.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_29/validation_metrics.json (2195 bytes; SHA 757be901e31f) - Validation partition performance and calibration data.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_47/architecture.json (184 bytes; SHA 3fb3df326afe) - Neural model architecture and dimensions.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_47/latency.json (228 bytes; SHA cbe8ea8bc8a4) - Model inference latency measurements in experimental environment.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_47/metadata.json (1623 bytes; SHA fbe820558d80) - Selected training/model configuration and artifact metadata.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_47/training_config.json (248 bytes; SHA cb52f843949c) - Seed-specific training hyperparameters and data configuration.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_47/training_history.json (6343 bytes; SHA 6e040bdf37f7) - Recorded neural training iterations and loss/metric history.

backend/models/experiments/neural_network_eurusd_m5_v1_seed_47/validation_metrics.json (2157 bytes; SHA e8b8b98a0364) - Validation partition performance and calibration data.

backend/models/registry/baseline_metadata.json (820 bytes; SHA 0177e1f30aa5) - Registered baseline-model summary.

backend/models/registry/hidden_markov_eurusd_m5_v1/checksums.json (379 bytes; SHA 70bc68c2408a) - Recorded integrity digests for registered artifact bundle.

backend/models/registry/hidden_markov_eurusd_m5_v1/evaluation.json (4421 bytes; SHA 3d8a25ca2c8e) - Stored model evaluation summary, not economic trade P&L.

backend/models/registry/hidden_markov_eurusd_m5_v1/metadata.json (1571 bytes; SHA 6a94f9c32b18) - Selected training/model configuration and artifact metadata.

backend/models/registry/hidden_markov_eurusd_m5_v1/parameters.npz (2643 bytes; SHA 4383de12c975) - Numerical arrays for HMM or normalization; not loaded in this review.

backend/models/registry/logistic_baseline.pkl (1551 bytes; SHA f6657b5d3dbf) - Serialized historical ML model; do not deserialize for inventory.

backend/models/registry/neural_network_eurusd_m5_v1/architecture.json (184 bytes; SHA 3fb3df326afe) - Neural model architecture and dimensions.

backend/models/registry/neural_network_eurusd_m5_v1/calibration.json (456 bytes; SHA d31d9bb82392) - Calibration coefficients/method selected for neural ensemble.

backend/models/registry/neural_network_eurusd_m5_v1/checksums.json (778 bytes; SHA 4fed30f2c29b) - Recorded integrity digests for registered artifact bundle.

backend/models/registry/neural_network_eurusd_m5_v1/evaluation.json (4181 bytes; SHA 351b19d26a9b) - Stored model evaluation summary, not economic trade P&L.

backend/models/registry/neural_network_eurusd_m5_v1/member_11.pt (1126661 bytes; SHA 25a52ae1bf0f) - Trained PyTorch neural-member weights; frozen binary, not executed.

backend/models/registry/neural_network_eurusd_m5_v1/member_29.pt (1126661 bytes; SHA ac219cae7725) - Trained PyTorch neural-member weights; frozen binary, not executed.

backend/models/registry/neural_network_eurusd_m5_v1/member_47.pt (1126661 bytes; SHA e5caf0cf035e) - Trained PyTorch neural-member weights; frozen binary, not executed.

backend/models/registry/neural_network_eurusd_m5_v1/metadata.json (1785 bytes; SHA 7d33fab0c74d) - Selected training/model configuration and artifact metadata.

backend/models/registry/neural_network_eurusd_m5_v1/normalization.npz (3786 bytes; SHA 9d1cfad522b8) - Numerical arrays for HMM or normalization; not loaded in this review.

backend/models/registry/xgboost_baseline.pkl (3886382 bytes; SHA bd2d485e0331) - Serialized historical ML model; do not deserialize for inventory.

backend/models/registry/xgboost_eurusd_m5_v1/evaluation.json (1605 bytes; SHA f21c948b811f) - Stored model evaluation summary, not economic trade P&L.

backend/models/registry/xgboost_eurusd_m5_v1/feature_importance.json (1175 bytes; SHA 6c4117b85548) - Model feature-importance values; not independent causal explanations.

backend/models/registry/xgboost_eurusd_m5_v1/metadata.json (983 bytes; SHA 4db2e059be9f) - Selected training/model configuration and artifact metadata.

backend/models/registry/xgboost_eurusd_m5_v1/metrics.json (635 bytes; SHA e1d95b8d189c) - Model training/validation metric summary.

backend/models/registry/xgboost_eurusd_m5_v1/model.pkl (3886382 bytes; SHA bd2d485e0331) - Serialized historical ML model; do not deserialize for inventory.

16.2 Archived project logs/ root files and audit scripts

logs/adaptation_baseline_20261027.json (51742 bytes; SHA a73a24ae74f3) - Dated audit, context or test result (top-level keys: captured_at, note, files); issuer/time trust not independently attested.

logs/adaptation_milestone_1_checkpoint_20261027.json (37421 bytes; SHA ae34a060daa9) - Dated audit, context or test result (top-level keys: captured_at, archive, archive_sha256, files, secret_files_included); issuer/time trust not independently attested.

logs/adaptation_milestone_1_source_20261027.zip (668410 bytes; SHA 0e6defd02512) - Dated source snapshot; archival evidence only, not the October 30 branch.

logs/adaptive_memory_historical_regression.log (9318 bytes; SHA 76570dca1a88) - Historical console output/test or audit log; not a current verification run.

logs/adaptive_memory_regression.log (518 bytes; SHA e57c1eae63a5) - Historical console output/test or audit log; not a current verification run.

logs/agents_folder_review_inventory.json (14593 bytes; SHA 0d59c3fa3b90) - Dated audit, context or test result (structured JSON array); issuer/time trust not independently attested.

logs/archive_clock_candidate_audit_20261027.json (771 bytes; SHA d37d6c56ab26) - Dated audit, context or test result (top-level keys: status, legacy_bars_scanned, candidate_offset_seconds, candidate_mapping_rejections, rejected_examples); issuer/time trust not independently attested.

logs/audit_archive_clock_candidates.py (2984 bytes; SHA 5f66aa0b3810) - Read every legacy M5 bar; analyze candidates without altering data/artifacts. Diagnostic/source utility; not executed for this document.

logs/audit_artifacts_20261027.json (1311 bytes; SHA e0ac352231fa) - Dated audit, context or test result (structured JSON array); issuer/time trust not independently attested.

logs/audit_backend_agents_test_agent_registry_20261027.log (3352 bytes; SHA 9c3cec3081ed) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_agents_test_base_agent_framework_20261027.log (1046 bytes; SHA eeaba0e2849e) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_agents_test_context_factory_macro_20261027.log (612 bytes; SHA 38fa79417b11) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_agents_test_core_market_agents_historical_20261027.log (3886 bytes; SHA 121cf74d0d22) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_agents_test_council_runner_20261027.log (1119 bytes; SHA 9f5547f254ef) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_agents_test_ecb_20261027.log (695 bytes; SHA 619965ad2450) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_macro_test_ecb_calendar_builder_20261027.log (798 bytes; SHA 73456c794a23) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_macro_test_ecb_calendar_protocol_20261027.log (656 bytes; SHA eaf471754c58) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_macro_test_ecb_rate_protocol_20261027.log (658 bytes; SHA 8dbba53bd99b) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_macro_test_fed_rate_protocol_20261027.log (978 bytes; SHA 190b9c60160f) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_main_20261027.log (490 bytes; SHA 20c631dddedc) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_market_state_test_liquidity_engine_20261027.log (2586 bytes; SHA eea650092b00) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_market_state_test_momentum_engine_20261027.log (1686 bytes; SHA d919de36fc51) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_market_state_test_session_engine_20261027.log (2919 bytes; SHA b98948cabd4d) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_market_state_test_structure_engine_20261027.log (2090 bytes; SHA 46518c70f778) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_market_state_test_trend_engine_20261027.log (1584 bytes; SHA 5935cf14d8ea) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_market_state_test_volatility_engine_20261027.log (1861 bytes; SHA b62bb5c951fb) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_models_test_dataset_partitioning_20261027.log (107 bytes; SHA bd751cbd5207) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_models_test_feature_builder_20261027.log (1033 bytes; SHA 7b87e4efab10) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_models_test_historical_dataset_builder_20261027.log (874 bytes; SHA da40e4715d61) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_models_test_model_contracts_20261027.log (555 bytes; SHA b9b6d293661b) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_models_test_model_registry_20261027.log (384 bytes; SHA ef0f59b29cec) - Historical console output/test or audit log; not a current verification run.

logs/audit_backend_models_test_neural_model_registry_20261027.log (1019 bytes; SHA 35c2c087c3b1) - Historical console output/test or audit log; not a current verification run.

logs/audit_blueprint_20261027.txt (7627 bytes; SHA 430ad30d19e5) - Historical source comparison, blueprint extraction or audit note.

logs/audit_database_20261027.json (19373 bytes; SHA 15864e964ee2) - Dated audit, context or test result (top-level keys: transaction, queries); issuer/time trust not independently attested.

logs/audit_dataset_readonly_20261027.log (1868 bytes; SHA 4c89323c34b9) - Historical console output/test or audit log; not a current verification run.

logs/audit_dataset_readonly_20261027.py (696 bytes; SHA 92357042c660) - Run the existing dataset integrity audit under enforced read-only SQL. Diagnostic/source utility; not executed for this document.

logs/audit_external_copies_20261027.json (2835 bytes; SHA 36d475890511) - Dated audit, context or test result (structured JSON array); issuer/time trust not independently attested.

logs/audit_fed_pytest_20261027.log (5770 bytes; SHA 5d00f7547982) - Historical console output/test or audit log; not a current verification run.

logs/audit_fed_pytest_verified_20261027.log (2756 bytes; SHA c82fb62c06f5) - Historical console output/test or audit log; not a current verification run.

logs/audit_inventory_20261027.json (195249 bytes; SHA 3847cf54922f) - Dated audit, context or test result (top-level keys: summary, files); issuer/time trust not independently attested.

logs/audit_project_20261027.log (31396 bytes; SHA c242cc7d957a) - Historical console output/test or audit log; not a current verification run.

logs/audit_project_20261027.py (4583 bytes; SHA 475cc1f6f828) - Read-only project inventory and bounded PostgreSQL audit; no broker access. Diagnostic/source utility; not executed for this document.

logs/audit_quality_events_20261027.py (851 bytes; SHA 514fb3760c57) - Historical audit, migration or verification script; source inventoried but not executed.

logs/audit_regressions_20261027.json (4327 bytes; SHA b75f088226ed) - Dated audit, context or test result (structured JSON array); issuer/time trust not independently attested.

logs/audit_regressions_20261027.py (2085 bytes; SHA bf4652b851b3) - Explicit safe regression allowlist; never run reserved evaluations. Diagnostic/source utility; not executed for this document.

logs/blueprint_review_inventory.json (222114 bytes; SHA a0985c8f1b40) - Dated audit, context or test result (structured JSON array); issuer/time trust not independently attested.

logs/canonical_council_tests.log (2858 bytes; SHA d01c4e61f459) - Historical console output/test or audit log; not a current verification run.

logs/canonical_dataset_check.log (1750 bytes; SHA 3d0004a66275) - Historical console output/test or audit log; not a current verification run.

logs/canonical_ecb_regression.log (1392 bytes; SHA b015dbb0f4a7) - Historical console output/test or audit log; not a current verification run.

logs/canonical_factory_check.log (1226 bytes; SHA ad517ef4dd1f) - Historical console output/test or audit log; not a current verification run.

logs/canonical_full_regression.log (2886 bytes; SHA 3be250eacecb) - Historical console output/test or audit log; not a current verification run.

logs/canonical_historical_integration.log (9318 bytes; SHA 12e6e38dad5c) - Historical console output/test or audit log; not a current verification run.

logs/canonical_legacy_runner.log (2240 bytes; SHA b09644f49d84) - Historical console output/test or audit log; not a current verification run.

logs/canonical_macro_context.log (1854 bytes; SHA e6233491ce4f) - Historical console output/test or audit log; not a current verification run.

logs/canonical_models_check.log (1112 bytes; SHA 4cca7f077a99) - Historical console output/test or audit log; not a current verification run.

logs/canonical_production_policy_result.json (795 bytes; SHA 4e00060e0c39) - Dated audit, context or test result (top-level keys: data_kind, registered, primary_success, primary_errors, primary_blocked); issuer/time trust not independently attested.

logs/canonical_registry_check.log (6706 bytes; SHA 6ca8bd2a7769) - Historical console output/test or audit log; not a current verification run.

logs/canonical_targeted_final.log (298 bytes; SHA be0c8636d907) - Historical console output/test or audit log; not a current verification run.

logs/clock_epoch_final_verification.json (809 bytes; SHA d2c320498a81) - Dated audit, context or test result (top-level keys: counts, health_as_of_last_capture, legacy_bars, legacy_latest_tick, frozen_model_artifacts_unchanged); issuer/time trust not independently attested.

logs/clock_epoch_historical_regression.log (9318 bytes; SHA d4f2bfa809de) - Historical console output/test or audit log; not a current verification run.

logs/clock_epoch_milestone_checkpoint_20261027.json (38933 bytes; SHA 6edb1df9d173) - Dated audit, context or test result (top-level keys: captured_at, archive, archive_sha256, files, clock_epoch_id); issuer/time trust not independently attested.

logs/clock_epoch_milestone_source_20261027.zip (696826 bytes; SHA f0f5fc974070) - Dated source snapshot; archival evidence only, not the October 30 branch.

logs/clock_epoch_regression.log (598 bytes; SHA fca052ab7c94) - Historical console output/test or audit log; not a current verification run.

logs/consolidate_context.py (5665 bytes; SHA ba4e12e65f10) - One-time, marker-checked relocation of existing input contracts. Diagnostic/source utility; not executed for this document.

logs/consolidate_registration.py (3127 bytes; SHA 7b89f9fd1e47) - Historical audit, migration or verification script; source inventoried but not executed.

logs/consolidate_research.py (1655 bytes; SHA fece011c5373) - Historical audit, migration or verification script; source inventoried but not executed.

logs/consolidation_context_tests.log (2246 bytes; SHA b7c5dd54ab0a) - Historical console output/test or audit log; not a current verification run.

logs/consolidation_initial_tests.log (126322 bytes; SHA 2d40d15ede9b) - Historical console output/test or audit log; not a current verification run.

logs/contracts_before_consolidation.py.txt (21430 bytes; SHA 014fe310ac0c) - Historical source comparison, blueprint extraction or audit note.

logs/council_analysis_context_before_consolidation.py.txt (10895 bytes; SHA 4e8fd06a45c8) - Historical source comparison, blueprint extraction or audit note.

logs/council_runner_before_consolidation.py.txt (7975 bytes; SHA 9f522bbc17a5) - Historical source comparison, blueprint extraction or audit note.

logs/council_runner_canonical.py.txt (9505 bytes; SHA 271f155b263d) - Historical source comparison, blueprint extraction or audit note.

logs/forward_feed_initial_counts.json (122 bytes; SHA c99c5951e294) - Dated audit, context or test result (top-level keys: oracle_utc_feed_epochs, oracle_utc_feed_captures, oracle_utc_ticks, oracle_utc_bars); issuer/time trust not independently attested.

logs/forward_feed_postgres_verification.json (257 bytes; SHA 8b20a1d41a5a) - Dated audit, context or test result (top-level keys: status, dialect, rolled_back, counts, legacy_tables_untouched); issuer/time trust not independently attested.

logs/forward_supervisor_integration.log (1383 bytes; SHA 36543c94abda) - Historical console output/test or audit log; not a current verification run.

logs/full_council_research_result.json (35737 bytes; SHA a014bd7a7257) - Dated audit, context or test result (top-level keys: analysis_mode, as_of, context_sha256, all_seats_completed, execution_authorized); issuer/time trust not independently attested.

logs/full_research_council_tests.log (388 bytes; SHA 9760a7017779) - Historical console output/test or audit log; not a current verification run.

logs/initialize_memory_store.py (1025 bytes; SHA 143d18231460) - Install only the five new, empty memory tables after rollback verification. Diagnostic/source utility; not executed for this document.

logs/memory_cli_1.json (36065 bytes; SHA 958f277048f4) - Dated audit, context or test result (top-level keys: analysis_mode, as_of, context_sha256, all_seats_completed, execution_authorized); issuer/time trust not independently attested.

logs/memory_cli_2.json (36066 bytes; SHA 4d0de90c064d) - Dated audit, context or test result (top-level keys: analysis_mode, as_of, context_sha256, all_seats_completed, execution_authorized); issuer/time trust not independently attested.

logs/memory_cli_first.json (0 bytes; SHA e3b0c44298fc) - Dated audit, context or test result (stored JSON record); issuer/time trust not independently attested.

logs/memory_cli_forecast_specs.json (177 bytes; SHA a6f2e8e21a2a) - Dated audit, context or test result (top-level keys: trend_master); issuer/time trust not independently attested.

logs/memory_cli_proof.json (662 bytes; SHA 05ff340cf20f) - Dated audit, context or test result (stored JSON record); issuer/time trust not independently attested.

logs/memory_cli_quotes.json (348 bytes; SHA 5bf874f887d1) - Dated audit, context or test result (structured JSON array); issuer/time trust not independently attested.

logs/memory_cli_settlement.json (1042 bytes; SHA 98ed1e021c8e) - Dated audit, context or test result (top-level keys: results, execution_authorized); issuer/time trust not independently attested.

logs/memory_postgres_installation.json (924 bytes; SHA 622f8c822eea) - Dated audit, context or test result (stored JSON record); issuer/time trust not independently attested.

logs/memory_postgres_verification.json (606 bytes; SHA f0ffc5a52cae) - Dated audit, context or test result (stored JSON record); issuer/time trust not independently attested.

logs/mt5_clock_diagnosis_20261027.json (574 bytes; SHA 171381e978e6) - Dated audit, context or test result (top-level keys: recorded_at, source, outside_sandbox_workers_connected, raw_timestamp_skew_approximately_seconds, timestamp_guard_blocked_ingestion); issuer/time trust not independently attested.

logs/mt5_time_domain_probe_20261027.json (5222 bytes; SHA d9c31b998a65) - Dated audit, context or test result (stored JSON record); issuer/time trust not independently attested.

logs/mt5_time_domain_probe_20261027_utf8.json (2697 bytes; SHA ec43a722243d) - Dated audit, context or test result (top-level keys: schema_version, read_only, initialized, observations, query_evidence); issuer/time trust not independently attested.

logs/original_blueprint_pdf_20261027.txt (8201 bytes; SHA 43d5c95b74c4) - Historical source comparison, blueprint extraction or audit note.

logs/prove_memory_cli.py (2612 bytes; SHA 8d2f98c612dc) - Historical audit, migration or verification script; source inventoried but not executed.

logs/register_core_agents_before_consolidation.py.txt (4158 bytes; SHA 83c694aafd49) - Historical source comparison, blueprint extraction or audit note.

logs/remaining_seats_complete_regression.log (2874 bytes; SHA 38204046d1f5) - Historical console output/test or audit log; not a current verification run.

logs/remaining_seats_factory_regression.log (1226 bytes; SHA ad517ef4dd1f) - Historical console output/test or audit log; not a current verification run.

logs/remaining_seats_fed_regression.log (2808 bytes; SHA 21fe35a46450) - Historical console output/test or audit log; not a current verification run.

logs/remaining_seats_historical_regression.log (7774 bytes; SHA c5a91eef796f) - Historical console output/test or audit log; not a current verification run.

logs/remaining_seats_registry_regression.log (6706 bytes; SHA 6ca8bd2a7769) - Historical console output/test or audit log; not a current verification run.

logs/remaining_seats_runner_regression.log (2240 bytes; SHA 3f25ba9eea52) - Historical console output/test or audit log; not a current verification run.

logs/remaining_seats_tests.log (316 bytes; SHA efe5f1bf5b36) - Historical console output/test or audit log; not a current verification run.

logs/research_council_before_consolidation.py.txt (6207 bytes; SHA f75bf6968051) - Historical source comparison, blueprint extraction or audit note.

logs/synthetic_council_memory.sqlite3 (225280 bytes; SHA 9e2783254c88) - Synthetic Council/memory proof database; not a live prediction history.

logs/synthetic_full_council_context.json (138289 bytes; SHA 921e732ab84e) - Dated audit, context or test result (top-level keys: symbol, timeframe, as_of, market_state, candles); issuer/time trust not independently attested.

logs/test_core_market_agents_before_consolidation.py.txt (10616 bytes; SHA dfa057d10df3) - Historical source comparison, blueprint extraction or audit note.

logs/test_core_market_agents_historical_before_consolidation.py.txt (14165 bytes; SHA f640b08ba145) - Historical source comparison, blueprint extraction or audit note.

logs/update_canonical_regression_expectations.py (3090 bytes; SHA 103ead5365ac) - Update legacy integration assertions to distinguish registered and blocked. Diagnostic/source utility; not executed for this document.

logs/verify_canonical_entrypoints.py (2356 bytes; SHA 3ccce8f0f579) - Save synthetic canonical entry-point evidence; no live acquisition or orders. Diagnostic/source utility; not executed for this document.

logs/verify_forward_feed_postgres.py (2203 bytes; SHA 0ddacfbd3a2f) - Historical audit, migration or verification script; source inventoried but not executed.

logs/verify_memory_postgres.py (2729 bytes; SHA e6422ee7f06d) - Historical audit, migration or verification script; source inventoried but not executed.

16.3 Repository root/build/credential metadata

.env - private runtime settings and secrets, known to be present in the archived ZIP; contents intentionally not inspected/reproduced.

.env.example - public names for DATABASE_URL, FRED_API_KEY and MT5_TERMINAL_PATH, without live secret contents.

requirements.txt / requirements-lock.txt - declared and locked Windows/Python package dependencies; do not equate a lock snapshot with reproducible installation proof.

get-pip.py - bundled third-party Python installer/bootstrap utility, not application strategy code.

.gitignore - exclusions for generated and private files; actual policy should be checked against current checkout.

.git/ - archived Git state refers to milestone/fed-seat25 at 85507c7...; this is **not** the later user-verified Stage 5 fb1c643 branch state.

venv/ - 894 bundled environment paths in the archive; not inventoried as application source.